

## Article

# Real-Time Detection and Response to Wormhole and Sinkhole Attacks in Wireless Sensor Networks

Tamara Zhukabayeva <sup>1,2</sup>, Lazzat Zholshiyeva <sup>1,2</sup> , Yerik Mardenov <sup>1</sup>, Atdhe Buja <sup>3</sup>, Shafiullah Khan <sup>4,\*</sup>  and Noha Alnazzawi <sup>5</sup>

<sup>1</sup> International Science Complex Astana, Astana 010000, Kazakhstan

<sup>2</sup> Department of Information Systems, L.N. Gumilyov Eurasian National University, Astana 010000, Kazakhstan

<sup>3</sup> Department of Mathematics, Computer Science, and Digital Forensics, Commonwealth University of Pennsylvania, Bloomsburg, PA 17815, USA

<sup>4</sup> College of Computer and Systems Engineering, Abdullah Al Salem University, Khaldiya 72303, Kuwait

<sup>5</sup> Department of Computer Science and Engineering, Yanbu Industrial College, Royal Commission for Jubail and Yanbu, Yanbu Industrial City 41912, Saudi Arabia

\* Correspondence: shafiullah.khan@aasu.edu.kw

## Abstract

Wireless sensor networks have become a vital technology that is extensively applied across multiple industries, including agriculture, industrial operations, and smart cities, as well as residential smart homes and environmental monitoring systems. Security threats emerge in these systems through hidden routing-level attacks such as Wormhole and Sinkhole attacks. The aim of this research was to develop a methodology for detecting security incidents in WSNs by conducting real-time analysis of Wormhole and Sinkhole attacks. Furthermore, the paper proposes a novel detection methodology combined with architectural enhancements to improve network robustness, measured by hop counts, delays, false data ratios, and route integrity. A real-time WSN infrastructure was developed using ZigBee and Global System for Mobile Communications/General Packet Radio Service (GSM/GPRS) technologies. To realistically simulate Wormhole and Sinkhole attack scenarios and conduct evaluations, we developed a modular cyber-physical architecture that supports real-time monitoring, repeatability, and integration of ZigBee- and GSM/GPRS-based attacker nodes. During the experimentation, Wormhole attacks caused the hop count to decrease from 4 to 3, while the average delay increased by 40%, and false sensor readings were introduced in over 30% of cases. Additionally, Sinkhole attacks led to a 27% increase in traffic concentration at the malicious node, disrupting load balancing and route integrity. The proposed multi-stage methodology includes data collection, preprocessing, anomaly detection using the 3-sigma rule, and risk-based decision making. Simulation results demonstrated that the methodology successfully detected route shortening, packet loss, and data manipulation in real time. Thus, the integration of anomaly-based detection with ZigBee and GSM/GPRS enables a timely response to security threats in critical WSN deployments.

**Keywords:** WSNs; security incident detection; wormhole; sinkhole; ZigBee; GSM/GPRS



Academic Editors: Sotirios K. Goudos, Silvia Rizzato and Antonino Proto

Received: 1 June 2025

Revised: 17 July 2025

Accepted: 1 August 2025

Published: 7 August 2025

**Citation:** Zhukabayeva, T.; Zholshiyeva, L.; Mardenov, Y.; Buja, A.; Khan, S.; Alnazzawi, N. Real-Time Detection and Response to Wormhole and Sinkhole Attacks in Wireless Sensor Networks. *Technologies* **2025**, *13*, 348. <https://doi.org/10.3390/technologies13080348>

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

The importance of wireless sensor networks (WSNs) in modern cyber-physical systems cannot be overstated as they are utilized in industrious facilities, smart cities, agriculture, healthcare, and in monitoring the environment. Regardless of their advantages, the architecture and the resources contained within WSNs their computing and

energy, as well as their dynamic topology are prone to a multitude of threats, especially network-level cyberattacks.

Wormhole, Sinkhole, Sybil, Denial of Service (DoS), Spoofing, and Man-in-the-Middle (MITM) attacks are frequently exposed to WSN infrastructures [1]. These attacks can compromise various components of the network infrastructure, such as routing paths, data integrity, and node availability, potentially leading to partial disruptions or complete system failure. Moreover, the integration of Internet of Things (IoT) devices has expanded the attack surface, increasing vulnerability [2]. WSNs face their most severe security threat from Wormhole attacks because these attacks establish hidden tunnels between malicious nodes to modify routing paths secretly. Through this method, the attack alters the routing path and timing of packets but maintains the packet content unaltered, which traditional security solutions cannot detect [3]. To counter complex routing-level attacks in WSNs, adaptive machine learning (ML) and deep learning (DL) methods are used to detect anomalies. Models like Support Vector Machines (SVMs), Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM), and Transformers show high accuracy in identifying malicious traffic in real time [4–6]. The research community investigates game theory-based [7] approaches, genetic algorithms [8], and energy-efficient trust models [9] as promising methods to reduce attack risks. The detection of cyberattacks benefits from heuristic methods [10] and ML-based traffic analysis techniques [11].

WSNs face a particular risk from Sinkhole attacks due to their distributed structure and resource limitations. An attacker executes Sinkhole attacks by taking control of either a single sensor network node or the base station to direct all information traffic toward the malicious destination. The network performance experiences severe degradation because this attack results in data interception, network disruption, reduced throughput, increased latency, and message loss [12]. Network security measures use trust-based mechanisms to identify Sinkhole attacks through the examination of node actions and communication behavior. These mechanisms generate trust ratings for nodes by analyzing parameters which include delivery ratios together with latency and energy consumption. Nodes which receive low trust scores become marked as suspicious entities. The first stage of a two-stage trust-based approach determines suspicious nodes, while the second stage validates their harmful actions. Sinkhole attack detection benefits from ML methods as one of its main detection approaches. The models learn data patterns to recognize both regular patterns along with security threats [13–15]. The implementation of WSN systems faces multiple obstacles when it comes to Sinkhole detection and prevention. The limited resource capabilities of sensor nodes in terms of energy and memory and computational power prevent the use of advanced algorithms [16]. The stability of trust models together with ML systems faces challenges when nodes continuously enter or exit WSN systems due to their dynamic network topology. Real-time attack detection in large-scale networks demands scalable and computationally efficient solutions [17]. Various strategies must be implemented to decrease energy usage in addition to the present ones. Sinkhole attacks require evaluation when compared to other routing protocols [18]. ZigBee demonstrates high energy efficiency and a flexible network topology, while GSM/GPRS provides wide area coverage and supports long-distance data transmission [19].

The combined implementation of these technologies enhances the real-time responsiveness to network security incidents [20–22]. The combination of these two communication systems enables networks to better handle security incidents through real-time response [23]. The WSNs operating on GSM/GPRS protocols ensure reliable data transmission for sensor nodes located in remote areas across extensive coverage zones [24].

The research focuses on developing a systematic approach for monitoring Wormhole and Sinkhole attacks in WSN systems which use ZigBee and GSM/GPRS technology for

real-time detection. The study involves constructing attack models followed by traffic and topology impact analysis and the development of new algorithms, together with architectural solutions to strengthen network dependability.

The aims of this research are as follows:

- To analyze the role of WSNs in cyber-physical systems and examine Wormhole and Sinkhole attacks.
- To develop a real-time detection method for these attacks.
- To experimentally validate the proposed approach.

The contributions are as follows:

- A real-time detection algorithm for early identification of Wormhole and Sinkhole attacks in WSNs is developed.
- Realistic Wormhole and Sinkhole attack scenarios are simulated using ZigBee and GSM/GPRS protocols.
- An anomaly-based detection method is implemented by comparing routing behavior under normal and attack conditions.
- Network performance metrics, including hop counts, latency, false data ratios, and route integrity, are evaluated.

The results show that our methodology successfully detects and mitigates Wormhole and Sinkhole attacks in real time, ensuring data integrity, minimizing packet loss, and maintaining stable network performance in WSN environments. The proposed framework achieves early detection with minimal energy and computational overhead, making it suitable for real-world WSN deployments in resource-constrained environments.

The rest of the paper is organized into five sections: Section 1—Introduction, Section 2—Related Work, Section 3—Methodology for Detection and Analysis of Security Incidents in WSNs, Section 4—Results, and Section 5—Conclusions.

## 2. Related Work

Due to constrained energy, computation, and storage capacities, WSNs are susceptible to multiple types of cyberattacks, particularly in the routing and application layers. Common routing-layer attacks include Sinkhole, Wormhole, Sybil, Spoofing, DoS, and MITM attacks [25,26]. These attacks often begin by manipulating routing protocols, such as redirecting traffic or injecting malicious paths, which weakens the network structure and facilitates subsequent compromises at the application layer, such as data falsification or unauthorized access to sensed information. As a result, not only are individual sensor nodes affected, but the integrity of the entire WSN infrastructure and its service to end users can be severely compromised.

Among the main types of attacks, Wormhole attacks pose a high-level threat to WSNs that operators find challenging to detect. Targeting WSNs, routing attacks focus on the routing protocols used within the system [27]. A common routing attack is the Black Hole attack, where a malicious node falsely advertises having the shortest path to the destination and subsequently drops all the intercepted packets [28]. The Gray Hole attack is similar, but selectively drops only some of the packets [29]. Bhatti D. S. et al. [30] explain that in a Wormhole attack, a tunnel is established between two malicious nodes, allowing them to bypass the standard routing path. The Sinkhole attack research by Faris, M. et al. [31] examines how a malicious node spreads false routing data to draw traffic from neighboring nodes. Network-layer attacks focus on the communication protocols which operate within wireless sensor network (WSN) systems. Hammi B. et al. [32] describe the Sybil attack as a method in which a malicious node forges multiple false identities to impersonate legitimate nodes, thereby disrupting the network. Similarly, the research conducted by De

Neira, A. B. et al. [33] explains how DoS attacks work to exhaust network resources and disrupt communication. Flooding attacks create network congestion through excessive data transmission [34], and Selective Forwarding attacks work by dropping critical packets selectively [35]. Physical-layer attacks focus on attacking the physical elements of wireless sensor networks. Jamming attacks interfere with wireless communication through the introduction of harmful signals into the network [36].

The Wormhole attack method establishes a pathway between two malicious nodes, which allows them to move information throughout the network beyond its normal communication route. The attackers gain control over the network traffic by modifying or dropping data packets, resulting in major communication failures. Firstly, Wormhole attacks distort routing, redirecting traffic incorrectly and leading to poor routing decisions and network fragmentation. Secondly, they compromise data integrity, as malicious nodes can alter or delete sensitive information, undermining network reliability. Thirdly, they lead to increased energy consumption, since repeated transmissions drain the energy of sensor nodes more rapidly. Similarly, the Sinkhole attack is also regarded as one of the most dangerous threats to WSNs, as it directly affects network routing, reliability, and data security. It is particularly stealthy, often bypassing traditional security mechanisms undetected [37–41].

Therefore, timely detection and analysis of security incidents in WSNs is essential to maintaining network integrity, and various methods have been proposed to detect such attacks. WSNs are increasingly adopting objective and systematic emergency response strategies that focus on attack prevention, rapid diagnosis, and effective mitigation to reduce system degradation. Trust-based schemes prevent attacks by excluding from the routing any malicious nodes that have low trust values. In WSNs, minimizing energy consumption is essential for sustaining the network over extended periods. Techniques for power optimization contribute to the long life of the batteries powering sensor nodes. For example, the method proposed by Shahid et al. [42] demonstrates 100% delivery and detection rates while maintaining low energy consumption. Protocol-specific methods often involve rule-based systems that classify traffic as normal or malicious. These systems can quickly identify anomalies without the computational burden of ML models when traffic is normal.

Table 1 presents a summary of the existing research landscape on IIoT, highlighting its advantages such as increased operational efficiency and greater cybersecurity via the incorporation of IDS into CPS with high precision. However, obstacles remain, such as the intricacies of technological integration, elevated redundancy expenses, and inadequate long-term data. The main goals continue to be reducing latency and boosting IIoT efficiency, while privacy, encryption, and scalability need thorough solutions.

**Table 1.** Comparative analysis of Wormhole attack detection methods.

| Method              | Approach                  | Description                                                                                                             | Accuracy | Authors                       |
|---------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------|----------|-------------------------------|
| Genetic Algorithms  | Optimization-based        | Used to optimize Wormhole detection parameters and improve effectiveness                                                | -        | Seidu R. et al. [43]          |
| MSAMAD              | Statistical/anomaly-based | Detects anomalous transmission patterns and one-hop delays between nodes using Mean Shift and Median Absolute Deviation | -        | Sun and Chen [44]             |
| Hop-Based Detection | Statistical               | Estimates the number of hops between nodes and identifies attacks if the number deviates significantly                  | 98%      | Yuan and Yan [45]             |
| EWATR Protocol      | Trust-based               | Energy-efficient protocol using trust and reputation metrics to detect Wormhole attacks                                 | -        | Alahmadi [46]                 |
| Hybrid LSTM + ELM   | Hybrid/ensemble           | Combines LSTM and Extreme Learning Machine to improve detection reliability                                             | 98.4%    | Sridevi and Anandan [47]      |
| EC-BRTT             | Real-time/time-based      | Measures round-trip time to detect Wormhole tunnels in real time                                                        | -        | Lakshmi Narayanan et al. [48] |

Table 2 presents various methods for detecting Wormhole attacks. Hop-Based Detection and Hybrid LSTM + ELM also demonstrate strong performance (98–98.4%). Although some methods lack detailed accuracy metrics, they offer advantages such as real-time detection, trust-based mechanisms, and efficient resource usage.

**Table 2.** Strengths, limitations, and efficiency of Wormhole attack detection techniques in WSNs.

| Method                  | Energy Consumption | Advantages                      | Disadvantages                  |
|-------------------------|--------------------|---------------------------------|--------------------------------|
| MSAMAD                  | Middle             | Effective anomaly detection     | Sensitive to latency           |
| Hop-Based Detection     | Low                | Simple and accurate             | Possibility of false positives |
| EWATR Protocol          | Middle             | Energy-efficient                | May produce false positives    |
| Hybrid LSTM+ ELM        | Low                | High accuracy                   | High accuracy                  |
| Genetic Algorithm-Based | Low                | Efficiency through optimization | Computational difficulty       |

Various WSN methods for attack detection demonstrate different performance levels in terms of accuracy alongside energy consumption and computational requirements. Researchers have investigated multiple detection methods to detect Sinkhole attacks. The Euclidean distance method calculates the distance between base stations and nodes to detect attacks [49]. Trust-based detection systems work by matching observed routing behavior against transmission patterns to detect malicious nodes [50]. An assessment revealed that the unified network and MAC layer data integration system decreased energy usage while providing better security control [51]. Power and memory and computational constraints in WSN devices restrict the operational use of robust algorithms [52]. The detection approaches need continuous monitoring combined with communication to work effectively, which leads to higher power usage and prevents their implementation in extensive and mobile network environments. The widespread application of certain methods requires additional hardware, real-time synchronization, and processes that heavily consume communication channels, which poses limitations for resource-constrained systems.

The security and data reliability of WSNs relies heavily on detecting Wormhole and Sinkhole attacks. The implementation of ML-based anomaly detection techniques alongside energy-efficient strategies demonstrated success in WSN systems, yet resource limitations and power requirements and scalability problems persist.

### 3. Methodology for Detection and Analysis of Security Incidents in WSNs

The comprehensive methodology we propose for detecting and analyzing security incidents in wireless sensor networks enhances the level of protection in WSNs.

This is achieved through the following objectives:

- (1) Maximizing the proportion of successfully detected incidents among all incidents that occur within a specific time interval,  $\Delta t$ .

$$\max_{m \in M} p_{\Delta}(I, m) \quad (1)$$

where  $p_{\Delta}(I, m)$  is the efficiency metric of incident detection using methodology  $m$  within a specific time interval,  $\Delta t$ , and where  $m \in M$  is the set of all considered methodologies.

- (2) Minimizing the number of false-positive security incidents detected within the  $\Delta t$  time interval:

$$\min_{m \in M} f_{\Delta t}(I, m) \quad (2)$$

where  $f_{\Delta t}(I, m)$  is the false-positive incident function or overall error function for methodology  $m$  within the time interval  $\Delta t$  and  $m \in M$  represents the set of methodologies aimed at minimizing this function.

- (3) Minimizing the communication and computational resource consumption of WSN nodes allocated for security incident detection and analysis, along with the overall costs required to achieve the objectives of the methodology.

To address these objectives simultaneously, a multi-criteria optimization problem is applied. It consists of the following stages:

- In the first stage, the solution space is narrowed using the Pareto optimization method, where the Pareto front is determined for the specific WSN configuration, i.e., the set of Pareto-optimal solutions that cannot be improved in one criterion without worsening another.
- A combined objective function is constructed by assigning weight coefficients based on the importance of various criteria in the second stage. The minimization of communication and computational resource usage by WSN nodes at discrete time points within the interval  $\Delta t$  is calculated as follows:

$$\min_{m \in M} \left\{ \frac{\max_{\Delta t \in T} \left( \max_{t \in \Delta t} |r_t^{(p)}| - \max_{t \in \Delta t} |r_t| \right)}{\max_{t \in \Delta t} |r_t|} \right\} \quad (3)$$

$$r^{(1)} = \text{imp} \left( r^{(0)} \right) \quad (4)$$

where

- $r_t$  is the resource consumption in normal operating mode (e.g., communication, computation, and energy);
- $r_t^{(p)}$  represents the resources used during incident processing;
- $\Delta t \in T$  is the set of discrete time intervals;
- $m \in M$  is the set of methodologies.

$\text{imp}$  is a function which modifies the initial resource vector,  $r^0$ , to produce an updated vector,  $r^{(1)}$ . The function performs a reassessment of resource distribution through the evaluation of security risks and threat elements.

This expression reflects the relative increase in resource load over the  $\Delta t \in T$  time interval for each methodology  $m \in M$ . In other words, it shows how much additional load is imposed on the system compared to normal consumption. The goal is to minimize this overhead, meaning to reduce the worst-case scenario of how much extra resource burden the incident detection process introduces into the system.

$T$  denotes the set of  $\Delta t$ . Each  $\Delta t$  represents a sequence of discrete time moments, based on which measurable indicators are calculated. The quantities  $r_t$  and  $r_t^{(p)}$  are normalized average vector values, adjusted using importance coefficients specific to the scenario. This normalization is performed within the range (0, 1), where 1 indicates maximum resource consumption.

The value of resource consumption is calculated as follows:

$$|r| = \left\| r^{(i+1)} \right\| = \sqrt{\left( r_n^{(i+1)} \right)^2 + \left( r_c^{(i+1)} \right)^2 + \left( r_e^{(i+1)} \right)^2} \quad (5)$$

where

- $r^{(i+1)}$  is the next iteration of the resource consumption vector;
- $r_n^{(i+1)}$  is the normalized value of the communication resource;
- $r_c^{(i+1)}$  is the normalized value of the computational resource;
- $r_e^{(i+1)}$  is the normalized value of the energy resource.

$$r_j^{(i+1)} = \frac{k_j \cdot r_j^{(i)}}{k_n + k_c + k_e}, \quad j \in \{n, c, e\} \quad (6)$$

Using this formula, the next iteration values of  $r_n$  (communication),  $r_c$  (computation), and  $r_e$  (energy resources) can be calculated. Here,  $k_j$  represents the importance coefficient of the respective resource, and  $k_n + k_c + k_e$  is the sum of all importance coefficients used for normalization. This approach allows for uniform processing of different resource types and facilitates automation of coding, analysis, and computation tasks.

The algorithm for transforming these coefficients is implemented based on known vulnerabilities of the hardware–software components of the wireless network. Such data can be obtained from open-source vulnerability databases, for example:

- Hardware and software characteristics receive identification through the Common Platform Enumeration (CPE);
- The Common Vulnerabilities and Exposures (CVEs) function to explain vulnerabilities;
- The Common Attack Pattern Enumeration and Classification (CAPEC) maintains a database that explains various attack patterns [53].

The system retrieves two specific values from the CAPEC database for every vulnerability which meets these particular requirements.

The database provides two specific values for every vulnerability which meets these specific criteria:

Likelihood\_Of\_Attack—this measurement determines the chance that an attack will occur; the Typical\_Severity evaluation determines the seriousness of the vulnerability. The classification system uses three levels of severity, which are High, Medium, and Low before converting them to 1, 0.66, and 0.33, respectively.

The methodology uses a one-time process to compute importance coefficients. In practical situations, organizations should extend the number of iterations to incorporate multiple vulnerability types and particular CPE matches.

The process determines the importance coefficient for communication resources by analyzing certain attack patterns: CAPEC-216 for communication channel manipulation, CAPEC-192 for protocol analysis, and CAPEC-117 for interception.

The analysis of computational resources determines the following importance coefficients: CAPEC-20 for encryption brute forcing, CAPEC-49 for password brute forcing, and CAPEC-97 for cryptanalysis.

The evaluation of energy resources leads to the following importance coefficients: CAPEC-604 for Wi-Fi jamming, CAPEC-124 for shared resource manipulation, and CAPEC-233 for privilege escalation.

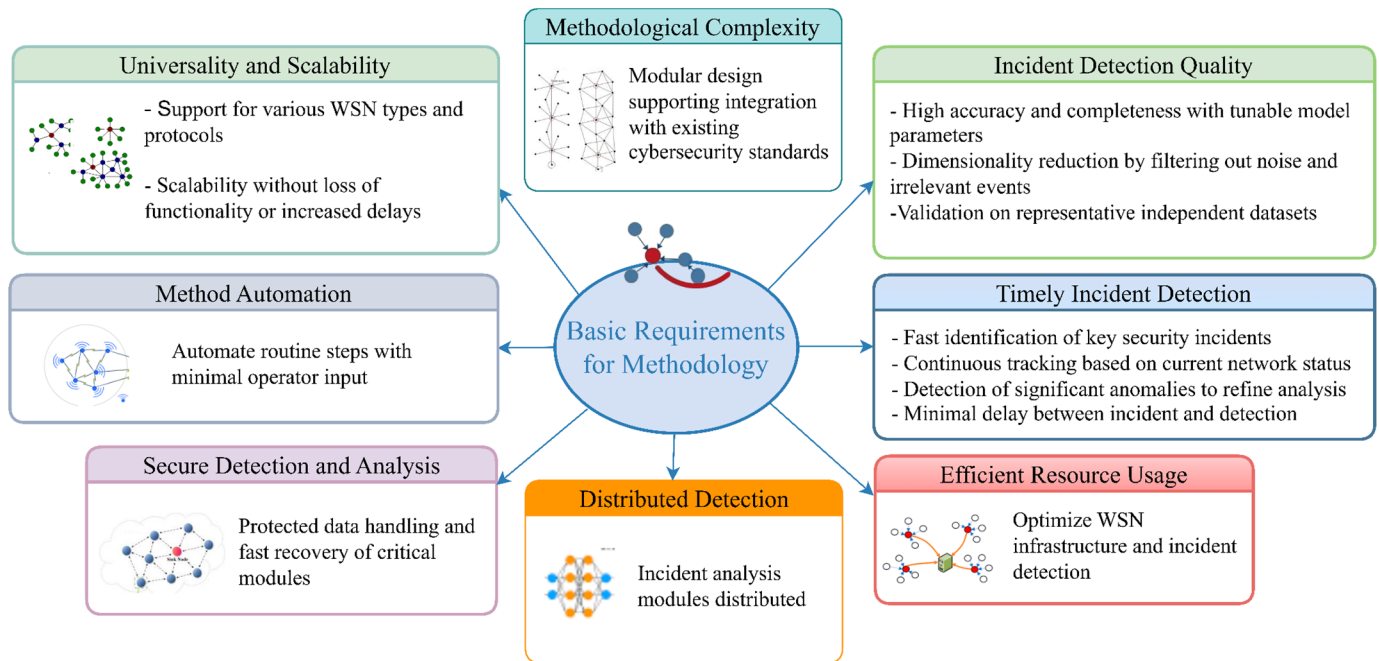
The determination of importance coefficients involves the use of specific mathematical equations:

$$\begin{aligned} k_n &= |V_n| \cdot \underline{P_n} \cdot \underline{S_n} \\ k_c &= |V_c| \cdot \underline{P_c} \cdot \underline{S_c} \\ k_e &= |V_e| \cdot \underline{P_e} \cdot \underline{S_e} \end{aligned} \quad (7)$$

where  $|V_n|$ ,  $|V_c|$ ,  $|V_e|$  represents the number of vulnerabilities related to communication, computation, and energy resources;  $\underline{P_x} - x \in \{n, c, e\}$  is the average attack likelihood (Like-

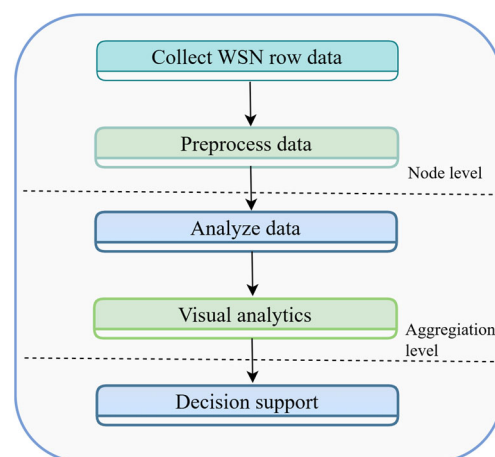
likelihood\_of\_attack) for the resource; and  $S_x$  is the average severity level (Typical\_Severity) of the corresponding vulnerabilities.

The basic requirements for the methodology are illustrated in Figure 1. These foundational elements represent essential capabilities that the proposed detection framework must fulfill to operate reliably and adaptively in real-world WSN environments.



**Figure 1.** Basic requirement types for the methodology.

The fulfillment of these requirements contributes to the development of a reliable and adaptable methodology for detecting and analyzing security incidents in WSNs. Such a methodology can adapt to evolving conditions and threats depending on the operational characteristics of specific WSNs. The main stages of the proposed methodology are illustrated in Figure 2.



**Figure 2.** Main stages of the methodology for detecting and analyzing security incidents in WSNs.

The proposed architecture is designed to be protocol-agnostic by operating primarily at the data abstraction layer. This means that regardless of the underlying communication protocol (e.g., ZigBee, Bluetooth, or GSM/GPRS), the system processes incoming data using a standardized preprocessing and analysis pipeline. Thus, the framework can integrate

data from heterogeneous WSNs, provided the data formats are converted to a compatible input structure.

Figure 2 shows the five main stages of the WSN security incident detection methodology in the form of an Algorithm 1. The initial step includes data collection through sensor and device networks to acquire both static and dynamic information. Dynamic data refers to time-varying parameters that are continuously collected from sensor nodes in real time (e.g., temperature, hop count, and packet arrival time). These data are cleaned and normalized during the pre-processing stage. Anomaly detection is performed based on the 3-sigma rule. During the experiments, the temperature sensor data were found to be the most reliable, as they showed clear deviations during attack (e.g., when changed to 80 °C by Spoofing) and closely matched the ground truth data. This is described in detail in Section 4.1.1. The following stage involves data preprocessing by cleaning and filtering and organizing for normalization to establish a single data format, i.e., data cleaning refers to the removal of corrupted, incomplete, or inconsistent data entries collected from sensor nodes. The results are analyzed visually to discover attack patterns. The implementation of countermeasures along with network operation restoration requires appropriate decision making. The implementation of these structured stages produces significant improvements in WSN security protection.

---

**Algorithm 1.** WSN Security Incident Detection Algorithm

---

Input:

$G = (V, E)$ —WSN graph, where  $V = \{R_1, R_2, \dots, R_n, C\}$  is the set of nodes and  $E$  is the set of network connections;

$D_{stat}$ —static data, including,  $CPE$ —protocols and security policies;

$D_{dyn}(t)$ —dynamic data that change over time (e.g., sensor readings, logs, and traffic).

Output:

- List of detected incidents;
  - Response actions;
  - Model-based conclusions and adjusted policies.
- 

**Step 1. Data Collection**

Static data:

$$D_{stat} = \{topology(R_i, C), CPE(R_i), policy(R_i, C)\} \quad (8)$$

Dynamic data:

$$D_{dyn}(t) = \{x_{R_i}(t), log_{R_i}(t), cmd_{R_i}(t)\} \quad (9)$$

**Step 2. Preprocessing**

Normalization:

$$x_{R_i}^{norm} = \frac{x_{R_i} - \mu_{R_i}}{\sigma_{R_i}} \quad (10)$$

Dimensionality reduction:  $Z = XW$ .

**Step 3. Incident Analysis**

Incident detection:

$$p_{\Delta t}(I, m) = \frac{|Detected\ Incidents_{R_i}|}{|Actual\ Incidents_{R_i}|} \rightarrow_{m \in M} max \quad (11)$$

Minimizing false alerts:

$$f_{\Delta t}(I, m) = \frac{|FalsePositives_{R_i}|}{|Total_{R_i}|} \rightarrow_{m \in M} \max \quad (12)$$

Resource consumption calculation:

$$|r| = \sqrt{\left(r_n^{(i+1)}\right)^2 + \left(r_c^{(i+1)}\right)^2 + \left(r_e^{(i+1)}\right)^2} \quad (13)$$

#### Step 4. Visual Monitoring and Analysis

Graph-based representation of routes  $R_i \rightarrow C$ .

$x_{R_i}(t) > \mu + 3\sigma$ —anomaly.

#### Step 5. Decision Making

Risk assessment:

$$Risk_{R_i} = P_{attack}(R_i) \cdot Impact_{R_i} \quad (14)$$

Action:

- Add a backup route for  $R_i$ ;
- Update policy from  $\pi_{R_i}$  to  $\pi_{R'_i}$ ;
- Analyze logs:  $\log_{R_i}(t)$ ,  $trace_{R_i}(t)$ .

Multi-objective optimization:

$$\begin{cases} P_{\Delta t}(I, m) \rightarrow \max \\ f_{\Delta t}(I, m) \rightarrow \min, \quad m \in M, R_i \in V, C \in V \\ |r| \rightarrow \min \end{cases} \quad (15)$$

The data obtained from WSN nodes are divided into static and dynamic data. Static data are values that do not change over time (node IDs, hardware characteristics, and initial configurations). Dynamic data are parameters that are constantly changing in real time (temperature, hop count, latency, etc.). Incident detection is based on the 3-sigma rule ( $\mu \pm 3\sigma$ ). The dynamic indicator represents the number of anomalies related to a particular attack type ( $R_i$ ). For example, in a Wormhole attack, the hop count increases sharply, while in a Sinkhole attack, the traffic is concentrated on a single node. Although ML methods are not used, predefined thresholds based on attack behavior are used. This makes the system efficient and lightweight in resource-constrained WSNs.

#### Threat model

The following capabilities are assumed for attackers: an attacker can capture one or more legitimate sensor nodes; two or more malicious nodes can interact and create a covert channel (e.g., a Wormhole tunnel); malicious nodes can route, modify, or drop packets; an attacker can attempt to manipulate traffic using false metrics (e.g., shortest path); an attacker does not control the entire network, but seeks to degrade performance or intercept data by strategically influencing network traffic. This threat model allows us to consider a wide range of attack scenarios.

The attacker is considered an internal node with legitimate cryptographic keys. For this reason, such attacks do not aim to break encryption algorithms but rather exploit weaknesses in routing logic and network behavior, which are often beyond the scope of cryptographic protection. For example, in a Wormhole attack, the attacker tunnels packets without decrypting them, thereby disrupting routing decisions. In a Sinkhole attack, the malicious node advertises attractive routing metrics to attract more traffic, but this does not violate the encryption rules.

We emphasize that unlike existing approaches, which either rely on resource-heavy ML/DL models or simulate attacks in purely software-based environments, our contribution integrates a multi-objective optimization framework with hardware-in-the-loop anomaly detection using ZigBee and GSM/GPRS in a modular cyber–physical system. This combination is novel in its lightweight, real-time response capability under constrained WSN conditions and does not require training data, making it highly suitable for real-world deployments.

Table 3 presents a comparative evaluation of existing routing attack detection methods for WSNs. The analysis reveals that traditional methods such as Hop-Based Detection and Rule-Based IDS offer lightweight, energy-efficient solutions but lack flexibility or robustness against sophisticated threats. ML approaches provide higher detection accuracy yet require intensive computation and training data, making them less suitable for deployment in constrained WSN environments. The proposed method integrates a hardware-in-the-loop design with multi-objective optimization and 3-sigma anomaly detection, enabling lightweight, real-time operation without the need for prior training.

**Table 3.** Comparison of detection methods for routing attacks in WSNs.

| Method              | Real-Time-Capable | Energy Efficiency | Computational Load | Remarks                                                                 |
|---------------------|-------------------|-------------------|--------------------|-------------------------------------------------------------------------|
| Hop-Based Detection | Yes               | High              | Low                | Limited to hop-based anomalies                                          |
| Hybrid LSTM + ELM   | Partial           | Medium            | High               | Requires offline training                                               |
| EWATR Protocol      | Partial           | Medium            | Medium             | Trust-based method; prone to false positives                            |
| Rule-Based IDS [54] | Yes               | High              | Low                | Fast and lightweight; not adaptive to evolving attack patterns          |
| Proposed            | Yes               | High              | Low                | Hardware-in-the-loop; no training needed; optimized for real-world WSNs |

## 4. Results

This section outlines the main findings achieved through the proposed methodology and consists of the following: a comparative analysis between regular path operations and Wormhole attack paths in WSNs; the development of a detection model for data alterations and losses in attack environments; the description of an anomaly detection method which identifies routing and data integrity attacks in WSN systems; the description of a detection algorithm which identifies anomalies within system attacks; an account of the establishment of Sinkhole attacks in WSNs together with their hardware–software architecture.

### 4.1. Algorithm

**Problem Statement:** Normal routing and Wormhole attack routing in WSNs should be compared while developing models which detect data manipulation and loss during attacks.

**Input Data:**

Set of nodes:  $R_4, R_3, R_2, R_1, R_0, C, R_5$ .

Normal communication links (edges):

$$E_{normal} = \{(R_4, R_3), (R_3, R_2), (R_2, R_1), (R_1, R_0), (R_0, C), (C, R_5)\} \quad (16)$$

Wormhole attack link (malicious channel):  $(R_4, R_5)$ .

#### Step 1: Node Identification and Graph Construction

A graph  $G = (V, E)$  is constructed, where all nodes and initial (normal) edges are added.

In normal mode, the shortest route is identified as  $R_3 \rightarrow R_2 \rightarrow R_1 \rightarrow R_0 \rightarrow C$ .

### Step 2: Sensor Data Generation

At each node, temperature values are simulated using a normal distribution,  
 $X \sim N(\mu = 25, \sigma = 2)$ :

$$x_i = \text{round}(N(25, 2), 1) \quad (17)$$

### Stage 3: Packet Transmission Modeling

Each intermediate node  $i$  forwards the packet to the next node.

In the absence of an attack, data are delivered unchanged (action = forwarded).

During a Wormhole attack:  $R_4$  manipulates the data and replaces them with false values ( $80.0^\circ\text{C}$ )  $\rightarrow$  Spoofing,  $R_5$  drops the packet in 30% of cases  $\rightarrow$  Denial of Service (DoS):

$$P(\text{drop}) = 0.3 \quad (18)$$

### Step 4: Route During Wormhole Attack

After the  $(R_4, R_5)$  additional edge is introduced, the shortest route changes to  $R_3 \rightarrow R_4 \rightarrow R_5 \rightarrow C$ .

This reduces the hop count from 4 to 3, and since ZigBee tends to select the shortest path, the malicious route is preferred.

#### 4.1.1. Modeling

Normal Sensor Data:

The temperature,  $x$ , at a given node is sampled from a normal distribution:  
 $X \sim N(25, 2)$ .

Spoofing:  $x_{\text{spoofed}} = 80.0$ .

DoS: A given packet is dropped under the following condition:

$$\text{if } r < 0.1 \Rightarrow \text{drop packet, where } r \sim \text{Uniform}(0, 1) \quad (19)$$

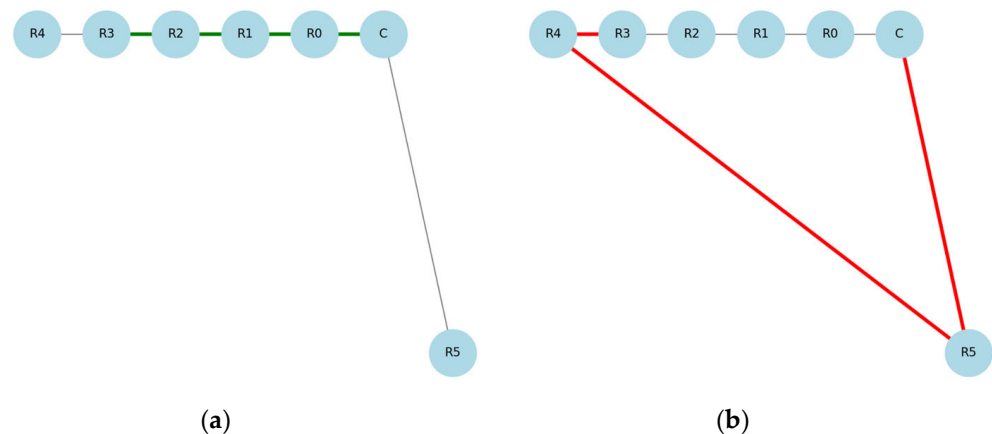
Route Length (hop count):

Under normal condition:  $h_{\text{normal}} = 4$ ;

Under Wormhole attack:  $h_{\text{attack}} = 3$ ;

Attack effectiveness:  $\Delta h = h_{\text{normal}} - h_{\text{attack}} = 1$ .

Figure 3 clearly illustrates how the attack affects routing and explains why the malicious path becomes attractive for the network.

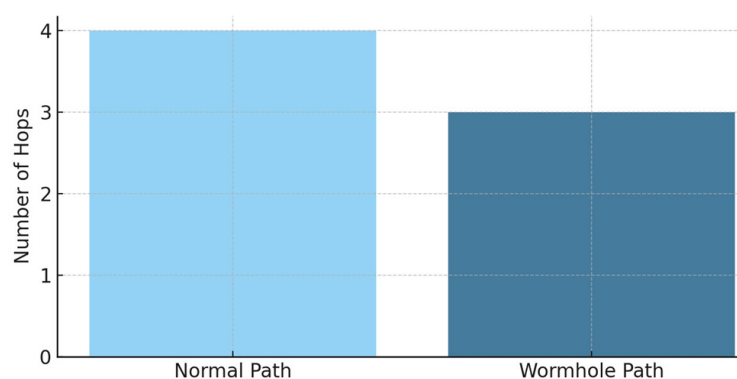


**Figure 3.** (a) Data transmission under normal conditions along the route  $R_3 \rightarrow R_2 \rightarrow R_1 \rightarrow R_0 \rightarrow C$  (green line). (b) The shortened route during a Wormhole attack:  $R_3 \rightarrow R_4 \rightarrow R_5 \rightarrow C$  (red line), where  $R_4$  and  $R_5$  are malicious nodes.

Research shows how Wormhole attacks impact routing functions and data assurance in wireless sensor networks. A network performance decline results when data interference occurs, which demonstrates the vital need for early detection of security threats.

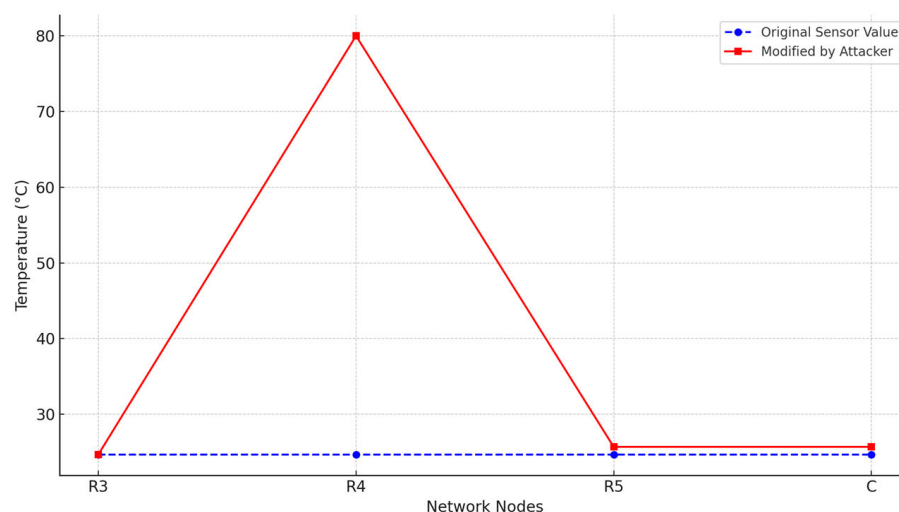
#### 4.1.2. Analysis

The researchers provide diagrams to analyze the Wormhole attack impact on network performance. Below, we present visualizations which display simulation outcomes that show routing path details and data integrity effects and help evaluate system reliability under hostile conditions. Figure 4 compares the hop count between the normal routing path and the path taken through the Wormhole. The reduction in hop count along the Wormhole path demonstrates how an attacker can influence decision making by exploiting cost metrics.



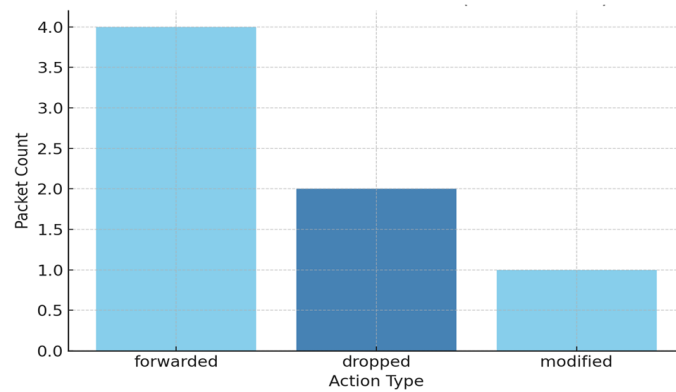
**Figure 4.** Comparative hop count in WSN routing.

Figure 5 compares the original and altered sensor readings during data transmission across the network. This demonstrates how the attacker at node *R4* can modify legitimate values and produce false outputs.



**Figure 5.** Sensor value integrity through Wormhole route.

Figure 6 presents a histogram of actions performed on data packets along the Wormhole route. This figure illustrates the frequency of packet forwarding, modification, and dropping actions to highlight the impact level of the attack.



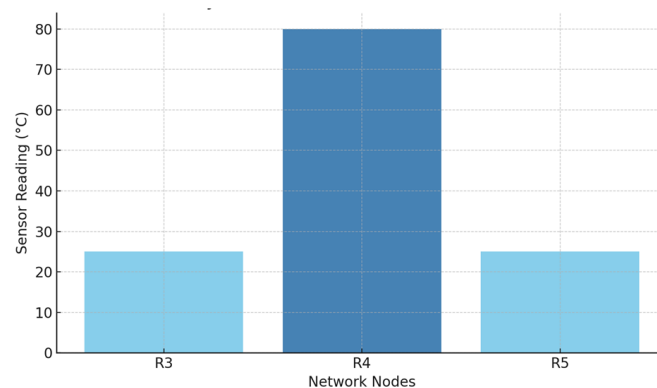
**Figure 6.** Packet handling frequency under Wormhole attack.

Reducing the hop count by 1 may seem small visually, but in a real network it makes a big difference. Nodes choose the shortest path, and the attacker receives more traffic. This leads to energy imbalance, data loss, network corruption, and topology instability. In the future, it is planned to expand the simulation to demonstrate these effects.

#### 4.2. Anomaly-Based Detection of Routing and Data Integrity Attacks in WSN Systems

Our study implemented a basic anomaly detection method to investigate Wormhole attacks in WSN systems. The detection mechanism works by searching for inconsistencies in packet delivery patterns and monitoring sensor information integrity. The detection method uses established security thresholds with specific domain heuristics to identify potential security breaches. The network experiment measured the temperature sensor readings of each node by using a Gaussian distribution with a mean of 25 °C and a standard deviation of 2 °C. The detection system uses the 3-sigma rule to identify anomalies because any value which surpasses three standard deviations from the mean gets marked as questionable. The researchers also developed a simulation to examine how malicious nodes would behave through data tampering actions along with selective packet elimination.

Through its simulation function, node *R4* takes on the role of a Spoofing agent that manipulates sensor readings to show 80 °C values. Node *R5* performs packet drops with a success rate of 30%, which imitates a DoS impact. Each data transmission action from node *R3* to the coordinator *C* is logged, recording the sender, receiver, data value, type of action, and whether an anomaly was detected. Figure 7 illustrates this route, with packets containing anomalies shown in red. The figure clearly distinguishes between normal transmissions and cases where data integrity violations were detected.



**Figure 7.** Anomaly detection on sensor transmission path.

Applying this detection model provides an important analytical capability for identifying threats in WSN infrastructures that are low-power, distributed, and utilize non-traditional security monitoring methods.

Figure 8 shows the detection of discrepancies. It presents three packet delivery activities recorded along the passage of coordinator C (R4 and R5) from node R3. The first and third broadcasts are labeled as forward, with related sensor readings of 24.7 °C and 24.8 °C, which is not known for general operation and no discrepancy. However, during the second transmission (R4–R5), a significant discrepancy is identified: the sensor datum was converted to 80 °C. The value is located outside the permissible range (as defined by the 3-sigma rule) and deliberately indicates data manipulation during a Wormhole attack.

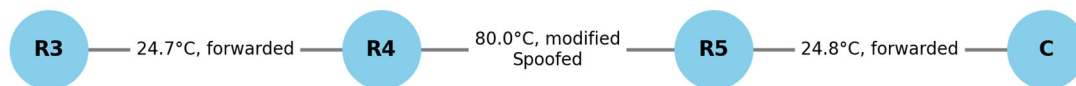


Figure 8. Anomaly detection log.

The following section details the algorithm used in this simulation, which has been used in WSNs and in discrepancy-based detection of data integrity attacks.

#### 4.3. Anomaly-Based Attack Detection Algorithm

**Aim:** To simulate Spoofing and packet-dropping attacks during sensor data transmission across the network and to detect anomalies based on the 3-sigma rule.

**Input data:**

Nodes—R3 → R4 → R5 → C;

Mean sensor value:  $\mu = 25.0$ ;

Standard deviation:  $\sigma = 2.0$ ;

Anomaly threshold:  $3\sigma$ .

**Model:**

**Sensor Data Generation**—The temperature value of each normal node is generated as follows:

$$x \sim N(\mu = 25, \sigma = 2) \quad (20)$$

**Anomaly Detection (3-Sigma Rule)**—A data point  $x$  is considered anomalous if

$$|x - \mu| > 3\sigma \Rightarrow |x - 25| > 6 \quad x \sim N(\mu = 25, \sigma = 2) \quad (21)$$

$x < 19$  or  $x > 31 \rightarrow$  anomaly.

**Spoofing**—R4 replaces the actual data with a false value:

$$x_{spoofed} = 80.0$$

$$|80 - 25| = 55 > 3\sigma \Rightarrow \quad (22)$$

**Dropping (DoS)**—R5 drops the packet with a 30% probability:

$$P(drop) = 0.3 \quad (23)$$

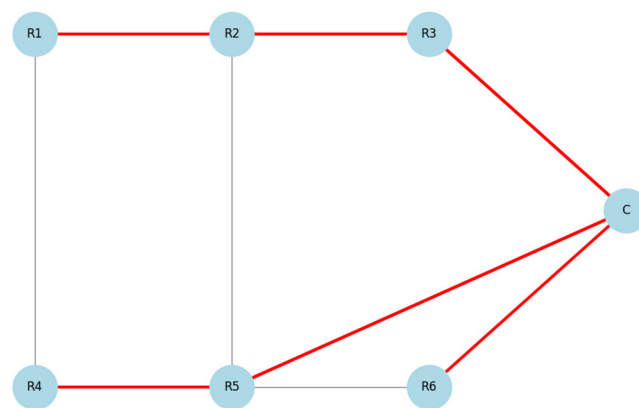
#### 4.4. Sinkhole Attack Modeling in a WSN System

A hidden tunnel is created between nodes R4 and R5, which allows data to bypass the standard ZigBee routing paths for the Wormhole attack. Node R4 sends corrupted data (for example, a temperature value of 80 °C), while node R5 randomly drops 30% of the packets. This attack creates a shortcut in the network topology, which leads to a reduction in the number of hops and a violation of the routing logic. Anomaly detection is performed

using the 3-sigma rule. During the Sinkhole attack, node R3 is configured to propagate false optimal route metrics, as a result of which most of the traffic from neighboring nodes is routed through this node. As a result, some of the redirected data are dropped, and the remaining packets are corrupted, simulating data corruption.

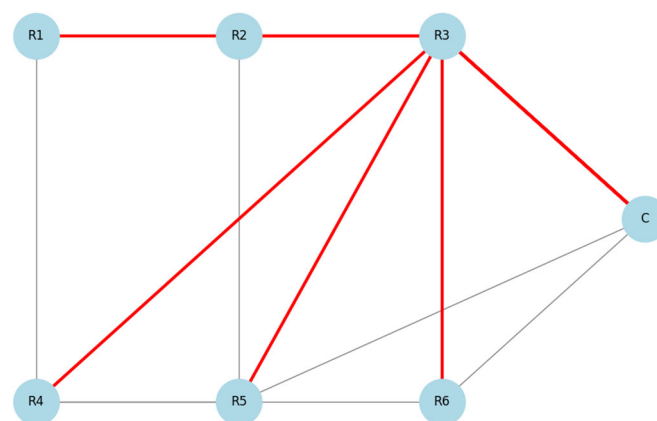
The simulation was performed in Python, and all network components are modeled as a directed graph. The attack parameters tunnel delay, packet drop probability, garbled values, and traffic redirection percentage were configured based on real WSN behavior patterns.

To model the impact of a Sinkhole attack in a WSN system, a network consisting of seven nodes (R1 – R6 and a coordinator, C) was constructed. Under normal conditions, each sensor node selects the most efficient route to reach the coordinator, C, using shortest-path algorithms. As shown in Figure 9, the routing paths are formed in various directions (e.g.,  $R1 \rightarrow R2 \rightarrow R3 \rightarrow C$  and  $R4 \rightarrow R5 \rightarrow C$ ), which helps balance the network load and prevents traffic concentration at a single point.



**Figure 9.** Routing paths before Sinkhole attack.

In an alternative scenario, during the attack simulation, compromised node R3 acts as a Sinkhole, broadcasting false information that makes it appear as the most efficient route to the coordinator. As a result, nodes R1, R2, R4, R5, and R6 begin routing their data through R3. This leads to a change in network topology (see Figure 10). Such a change concentrates network traffic around the malicious node, allowing the attacker to either drop or modify the packets.



**Figure 10.** Routing paths after Sinkhole attack (R3 is malicious).

Table 4 indicates that under normal conditions, all packets were successfully broadcast, and the recorded temperature values remained within the expected range (22.0–26.4 °C), demonstrating reliable and stable data transmission. For comparison, the table also presents

scenarios involving various malicious activities. For instance, the coordinator ID from node R2 was spoofed in packets relayed through node R3 and replaced with a falsified temperature reading of 80.0 °C. Furthermore, packets originating from nodes R1, R4, R5, and R6 were dropped, either after passing through the compromised node or as a result of being routed in that direction.

**Table 4.** Pre-attack and post-attack transmission logs.

|   | Source | From | To         |             | Data       |             | Action     |             |
|---|--------|------|------------|-------------|------------|-------------|------------|-------------|
|   |        |      | Pre-Attack | Post-Attack | Pre-Attack | Post-Attack | Pre-Attack | Post-Attack |
| 0 | R1     | R1   | R2         | R3          | 24.7       | 25.2        | Forwarded  | Forwarded   |
| 1 | R1     | R2   | R3         | C           | 24.7       | NaN         | Forwarded  | Dropped     |
| 2 | R1     | R3   | C          | R3          | 24.8       | 27.3        | Forwarded  | Forwarded   |
| 3 | R2     | R2   | R3         | C           | 26.4       | 80.0        | Forwarded  | Modified    |
| 4 | R2     | R3   | C          | R3          | 24.7       | 27.7        | Forwarded  | Forwarded   |
| 5 | R4     | R4   | R5         | C           | 22.0       | NaN         | Forwarded  | Dropped     |
| 6 | R4     | R5   | C          | R3          | 25.7       | 23.1        | Forwarded  | Forwarded   |
| 7 | R5     | R5   | C          | C           | 24.5       | NaN         | Forwarded  | Dropped     |
| 8 | R6     | R6   | C          | R3          | 24.6       | 22.1        | Forwarded  | Forwarded   |

Overall, the simulations were successful and clearly demonstrated how a Sinkhole attack disrupts the routing logic of network nodes.

#### The Sinkhole attack simulation algorithm

Undirected graph representing the WSNs:

$$G = (V, E) \quad (24)$$

where  $V$  is the set of nodes and  $E \subseteq V \times V$  is the set of edges.

The coordinator node— $C \in V$ ;

Set of sensor nodes (data sources)— $S \in V : S = \{R_1, R_2, R_3, R_4, R_5, R_6\}$ ;

Malicious node— $v_{sink} = R \in V$ ;

Routing path from source node  $s \in S$  to the sink  $C : P_s$ .

Sensor data value  $d \in R$ .

Transmission log (actions performed on packets)— $A$ .

#### Step 1: Routing Before Attack

For each source  $s \in S$ , determine the shortest path to the sink using:

$$P_s^{before} = \text{Shortest}(G, s, C) \quad (25)$$

#### Step 2: Routing After Sinkhole Attack

After the attack, all source nodes (except the attacker itself) reroute traffic through the Sinkhole node:

$$P_s^{before} = \begin{cases} [s, v_{sink}, C], & \text{if } s \neq v_{sink} \\ \text{ShortestPath}(G, s, C), & \text{if } s = v_{sink} \end{cases} \quad (26)$$

#### Step 3: Sensor Data Generation

Sensor readings are simulated as normally distributed values:

$$d \sim N(25, 2) \quad (27)$$

#### Step 4: Packet Transmission with Attack Behavior

For each path  $P_s = [v_1, v_2, \dots, v_n]$ , transmit packets along each link  $(v_i, v_{i+1})$  and log the action:

$$A = \begin{cases} \text{dropped}, & \text{if } v_i = v_{\text{sink}} \text{ and } v_{\text{drop}} < 0.4 \\ \text{modified}(d = 80.0), & \text{if } v_i = v_{\text{sink}} \text{ and } v_{\text{modify}} < 0.5 \\ \text{forwarded}(d), & \text{otherwise} \end{cases} \quad (28)$$

where

$P_{\text{drop}} \sim u(0, 1)$  is the probability of dropping the packet;

$P_{\text{modify}} \sim u(0, 1)$  is the probability of modifying the packet.

**Step 5:** Transmission Log Format

Each transmission is recorded as

$$A = \{ (s, v_i, v_{i+1}), d, \text{action} \} \}_{\forall (v_i, v_{i+1}) \in P_s, \forall s \in S} \quad (29)$$

where  $\text{action} \in \{\text{forwarded}, \text{modified}, \text{dropped}\}$ .

The proposed simulation model clearly demonstrated the impact of a Sinkhole attack on WSN network topology and data routing. Under normal conditions, each node selects the shortest path to the coordinator, enabling balanced traffic distribution and reliable data exchange. However, during a Sinkhole attack, the malicious node (R3) falsely advertises itself as the optimal route, redirecting the traffic of other nodes toward itself. Simulation results showed that nodes altered their routing paths, transmitting data centrally through R3. This situation gives the attacker the ability to drop packets (DoS) or replace them with false values (Spoofing). The model demonstrated that such an attack poses a direct threat to network reliability and data integrity. Thus, to detect and prevent such attacks in WSN infrastructures, there is a critical need for implementing adaptive routing, analysis-based anomaly detection algorithms, and reinforced security policies. Moreover, the obtained results can serve as a baseline for testing defense mechanisms against Sinkhole attacks.

#### 4.5. Hardware–Software Implementation

##### 4.5.1. Justification of Architectural Decisions and Comparison of Alternative WSN Attack Modeling Methods

In the context of modeling network attacks in WSNs, it is crucial to create an experimental environment that closely resembles real operating conditions to ensure the reliability of research results [55]. The architecture developed in accordance with these requirements is a modular cyber–physical system focused on flexibility, scalability, and the reproducibility of attack scenarios.

The components and logical structure were selected based on the following factors:

- The use of the ZigBee protocol, based on the IEEE 802.15.4 standard, ensures realistic data transmission and support for self-organizing networks and reflects typical features of industrial WSNs [56].
- The combined use of Raspberry Pi single-board computers and Arduino Uno micro-controllers allows for the creation of not only simple sensor nodes but also intelligent attacker nodes with computational capabilities.
- Dividing the system into sensor, network, and analytic layers facilitates traceability, simplifies error analysis, and enables easy system expansion when adding new components or attack types.
- A realistic Wormhole attack scenario is implemented by establishing a covert channel over GSM/GPRS between distant nodes within a standard ZigBee network. This approach allows for realistic attacker behavior modeling and creates a type of attack that is not easily detectable using standard methods.

The integration of ZigBee and GSM/GPRS technologies in this study is driven by specific experimental and architectural requirements. ZigBee, standardized under IEEE 802.15.4, remains a popular choice for wireless sensor networks (WSNs) due to its low power consumption, self-organizing capabilities, and suitability for short-range communication [57]. GSM/GPRS is instead used to emulate a covert channel between two distant attacker nodes as a critical requirement for realistic Wormhole attack modeling. GSM/GPRS continues to serve wide-area reach and attack emulation applications, even though some implementations plan to discontinue its use [58]. This approach utilizes GSM/GPRS to conduct experimental evaluations which employ ZigBee nodes as valid network elements while treating the GSM/GPRS as an attacker channel bypass. The testing framework avoids using LoRaWAN or NB-IoT because these solutions require complicated hardware integration and add significant complexity to modular testbed systems [59,60].

Thus, the selected architecture is reproducible and has been experimentally verified, making it a reliable platform for research on WSN security. To justify the suitability of this architectural decision, now we examine alternative methods for modeling of WSN attacks.

Table 5 summarizes key approaches for modeling WSN attacks, comparing software simulators and hardware-based testbeds. Software platforms like NS-3, OMNeT++, and Cooja allow quick experimentation but lack realism in protocol behavior and hardware effects. Arduino- or ESP32-based testbeds offer physical deployment and low cost but struggle to support complex attacks due to limited resources. Hybrid setups combining Arduino and Raspberry Pi provide a balance between realism and computational power, though they introduce integration challenges.

**Table 5.** Comparative analysis of methods for modeling WSN attacks.

| Method                               | Description                                                                  | Advantages                                                     | Disadvantages                                                                                                      |
|--------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Simulators NS-3, OMNET++, Cooja      | Software-based modeling of nodes and attacks                                 | Rapid model development without the need for physical hardware | Limited realism of protocol modeling; electromagnetic and hardware-level factors cannot be evaluated               |
| Arduino/ESP32 Hardware-Based Testbed | Implementation of a physical network with limited resources                  | Low-cost and simple                                            | Difficult to implement complex attacks (e.g., Wormhole and Sybil attacks) without additional computational modules |
| Arduino + Raspberry Pi               | A hybrid system that supports both simple and logic-intensive attacker nodes | Balance between cost, computational power, and realism         | Difficulty in component integration and configuration                                                              |

#### 4.5.2. Hardware–Software Architecture

To ensure realistic attack modeling and accurate evaluation of network behavior, the hardware–software architecture was designed and implemented by the authors as a modular cyber–physical system, which is described in this section. This architecture was selected based on its ability to balance real-time monitoring, repeatability, and compatibility with both low-power sensor nodes and intelligent attacker nodes using ZigBee and GSM/GPRS technologies.

##### The overall structure of cyber–physical systems

In this study, a cyber–physical system is an integrated, distributed infrastructure which integrates hardware and software components to monitor environmental parameters, collecting data and later conducting analyses (Figures 11 and 12).

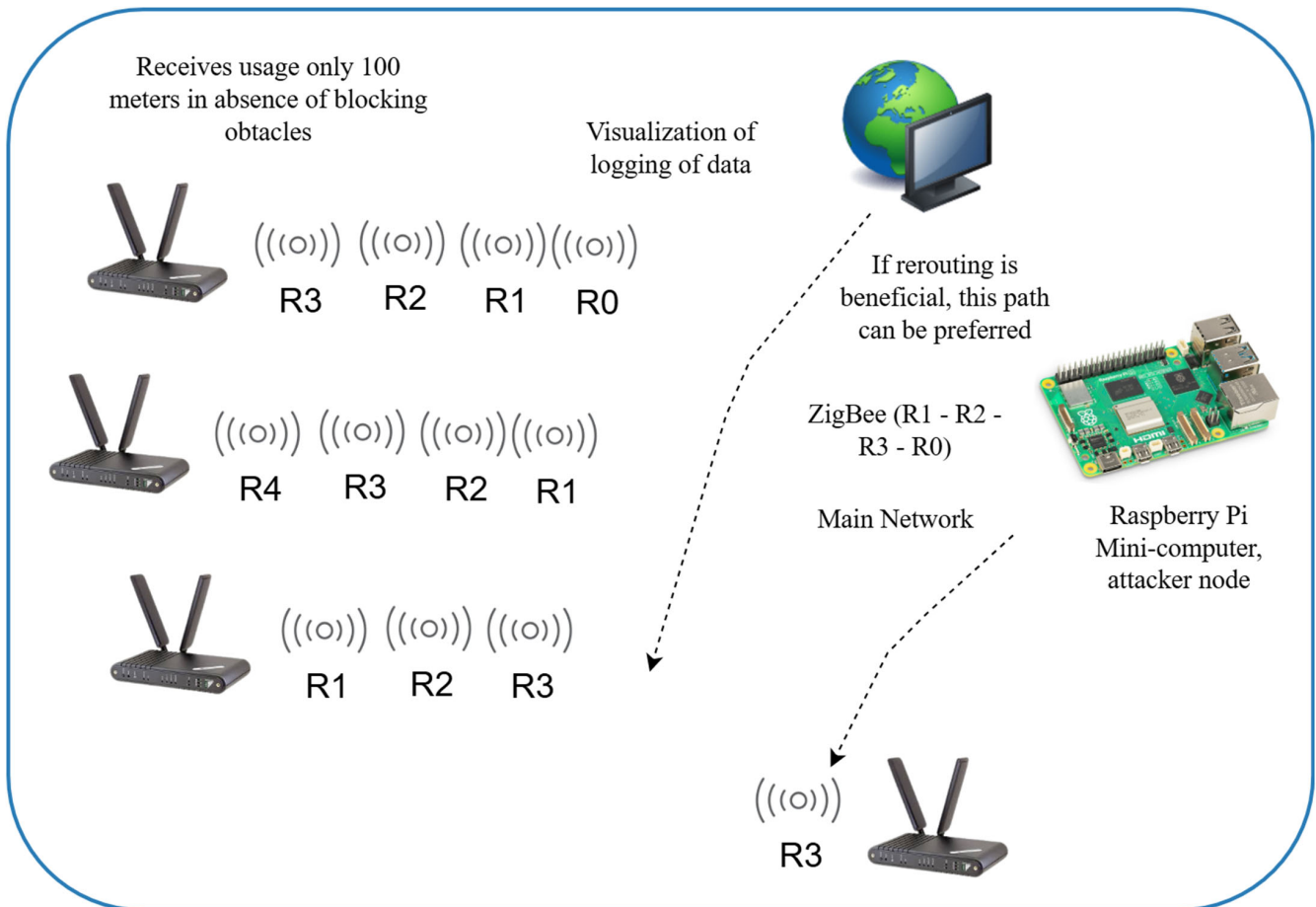


Figure 11. Cyber-physical system.

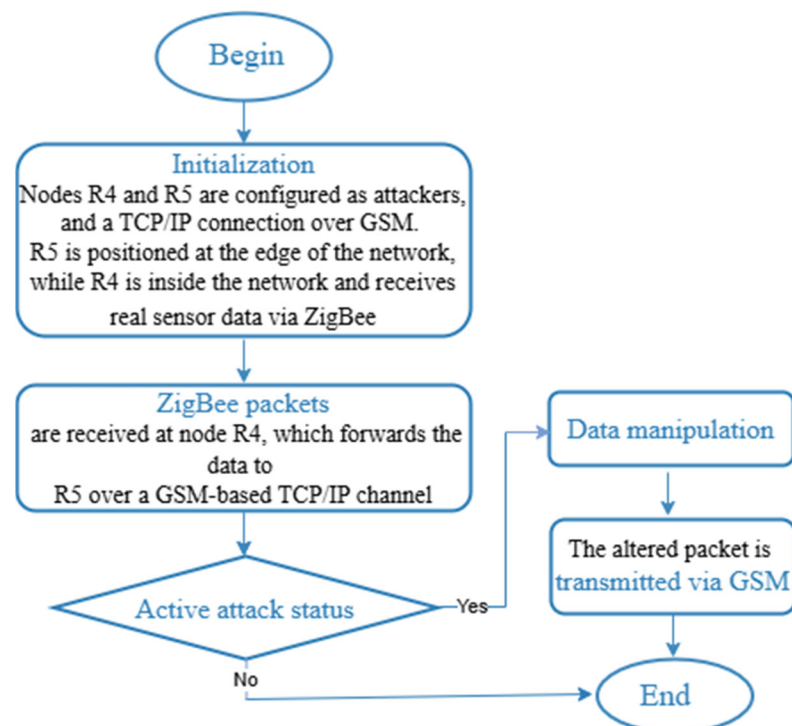


Figure 12. Wormhole attack modeling algorithm.

The system is designed to simulate actual processes in WSNs and to repeat network-level attacks, as well as to capture and analyze their effects.

The architecture is built on a modular theory and includes three main layers: a sensor node layer, a data transmission layer (WSN), and an information processing and visual layer.

This allows for clear differences in structural separation component functions and the implementation of various operating modes and analyses of the behavior of the system.

At the first level, sensor nodes operate autonomously. Each node consists of a microcontroller (Arduino Uno or Raspberry Pi), one or more physical sensors, a wireless communication module (XBEE Series 2), and a power supply. These sensors measure environmental parameters such as temperature, humidity, and pollutant concentrations (e.g., carbon monoxide, methane, dust, etc.). The nodes are configured to transmit data to the network at predefined time intervals.

At the second level, nodes communicate with each other using the ZigBee protocol. Thanks to the embedded logic within the XBEE modules, the network is self-organizing, enabling automatic address assignment, routing, and message distribution without centralized control. This study adopts a linear topology, where each node communicates only with its immediate neighbors. This setup facilitates precise monitoring of routing behavior, particularly when targeting specific network segments. The network includes one coordinator and six routers. The coordinator is connected to a computer via USB and is responsible for managing the network and collecting data.

At the third level, data reception, processing, and visualization are performed. A personal computer connected to the coordinator via a USB interface receives the sensor data, logs them, displays them in real time, and analyzes the network status. A dashboard interface is employed to visualize sensor readings, network topology, and error or anomaly messages.

This architecture enables a complete operational cycle from physical environmental measurement to the analysis of network behavior under both normal and anomalous conditions.

At the network interaction level, R4 and R5 attacks are simulated using routers equipped with Raspberry Pi boards and GSM/GPRS modules. A covert communication channel is established between distant nodes, allowing malicious traffic to bypass legitimate routing paths. This method exposes vulnerabilities in the routing mechanism and evaluates the impact of such attacks on network security. As a result, the system becomes a viable platform for modeling the behavior of wireless sensor networks (WSNs), simulating various attack scenarios, and testing defense algorithms.

### **WSN Node Configuration**

The WSN configuration consists of seven nodes, each fulfilling distinct roles in data collection, transmission, and processing. These nodes vary in their hardware setups, functional responsibilities, and communication capabilities. The nodes are categorized into two main types: coordinators and routers.

The coordinator is responsible for initial configuration, network setup, and overall management. It is connected to a personal computer via an XBEE USB adapter and operates in continuous reception mode, receiving data from other nodes in the network (Figure 13).

The coordinator uses a Digi XBEE Series 2 module, which is configured in API firmware mode. This configuration enables the following functionalities: setting network parameters and assigning a PAN ID (Personal Area Network Identifier), managing the routing table, and ensuring stable and reliable communication.

Data received by the coordinator are transmitted to a dedicated software application, where they are visualized and logged into system files.

The remaining six nodes function both as routers and sensor modules, enabling data forwarding as well as direct environmental data collection. Nodes R0, R1, R2, and R3 are

built on Arduino Uno microcontrollers, with each node equipped with an XBEE module operating in router API mode. These nodes are outfitted with various sensors capable of measuring multiple environmental parameters, including temperature (via the DS18B20 thermometer), humidity (using DHT11 or DHT22 sensors), atmospheric pressure (with the BMP180 sensor), and air pollution (detected by MQ-7, MQ-135, and optional sensors for CO and CH<sub>4</sub> gases). Additionally, the nodes provide an overall air quality index based on the collected data (Figure 14).

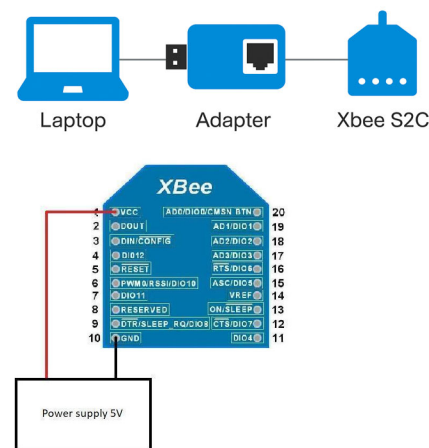


Figure 13. WSN coordinator node diagram.

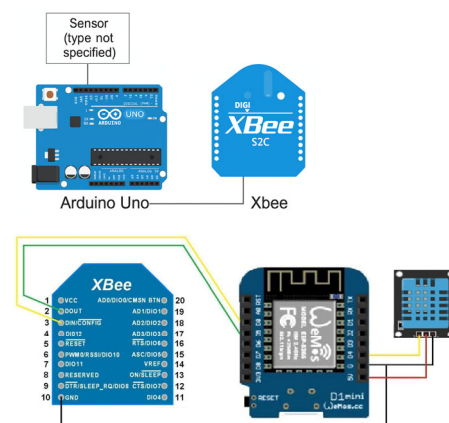
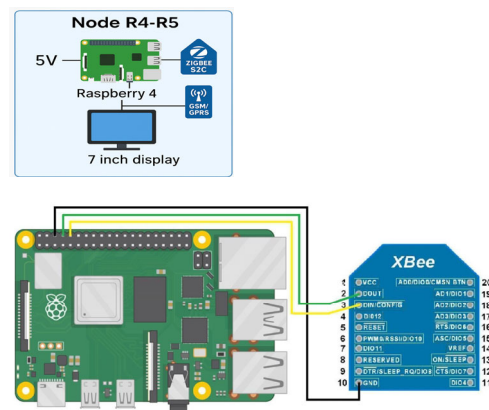


Figure 14. Diagram of standalone nodes for traffic forwarding.

The sensors are connected through the standard Arduino interface, utilizing both digital and analog input pins. Measurements are taken at defined time intervals (e.g., every 1 s). An Arduino sketch implements an algorithm to read, format, and transmit the sensor data as a packet to the coordinator address. Data transmission is managed by the XBee module, which automatically selects the most efficient route to the coordinator based on the current network topology.

Nodes R4 and R5 serve as extended routers and include a Raspberry Pi single-board computer (version 3 or higher) (Figure 15). These nodes are also equipped with XBee modules and GSM/GPRS modules (e.g., SIM800 or SIM900). They are specifically designed to simulate Wormhole-type attacks. The Raspberry Pi devices establish a TCP/IP connection over an external mobile network between the two nodes.



**Figure 15.** Diagram of extended router (R4, R5).

Custom Python 3.12.3 software used on these nodes performs the following functions:

Receives Zigbee frames via the XBee module;

Encapsulates the data in TCP packets and transmits them over the GSM/GPRS connection.

On the receiving node, TCP packets are received, the Zigbee data are extracted, and then retransmitted into the Zigbee network via the XBee module.

A covert communication channel is established that remains undetected within the standard Zigbee topology. This channel can be exploited to intercept, modify, or block network traffic. Such nodes play a crucial role in identifying network vulnerabilities and evaluating the system's resilience to various types of attacks.

Each node in the network is powered independently either via battery or a connected power adapter, allowing for autonomous operation. This setup better reflects real-world deployment scenarios, where WSN elements may be installed in remote or difficult-to-access areas or for extended periods. The hardware and software components of each node are tightly integrated.

For microcontroller-based nodes, the XBEE Arduino Library is used to manage the XBEE module. For nodes based on the Raspberry Pi, the Digi-XBee Python Library is utilized. The system supports firmware updates, reconfiguration of network parameters, and sensor calibration. Additionally, nodes are capable of autonomously restoring communication after temporary failures a critical feature for functioning in unstable or volatile environments.

### Communication protocols and node roles used in the system

The hardware–software WSN system employs Zigbee as the primary wireless communication protocol, implemented using Digi XBee Series 2 modules. Zigbee, based on the IEEE 802.15.4 standard [61], is designed to support self-organizing, fault-tolerant, scalable, and low-power networks, making it ideal for control and monitoring applications.

Zigbee follows a hierarchical network topology and supports three types of devices:

**Coordinator**—This serves as the central node of the network. It is responsible for initial setup, address allocation, and routing management. In the current configuration, the coordinator is connected to a PC via an XBee-USB adapter. It collects data from sensor nodes, logs them, transmits them to a monitoring dashboard, and oversees overall network performance.

- **Router**—Intermediate nodes which relay packets between pieces of end equipment and coordinators. They can also collect data from sensors. In an experimental setup, nodes R0 and R5 serve as routers. They are important for routing and traffic analysis, especially during Wormhole attack modeling, where the attackers try to replace the data delivery path.

- **Final Equipment**—Inactive participants in the network who connect to the router and do not relay traffic. These devices are not used in this test, because each node should support retrosion to facilitate safety analysis and complex routing. To apply the Wormhole attack, a TCP/IP protocol is used as an additional communication channel between nodes *R4* and *R5*, operated on GSM/GPRS.

Figure 16 illustrates the hardware components used in the experimental setup. Each Arduino-based node is connected to an XBee module via UART (TX/RX) and powered either through a USB connection or an external battery. These nodes are equipped with various sensors, including temperature, humidity, and air quality sensors.



**Figure 16.** Hardware component.

The Raspberry Pi is connected to an XBee module through either a USB adapter or the GPIO interface and to a GSM/GPRS module (e.g., SIM800 or SIM900) via direct connection. All seven nodes in the wireless sensor network (WSN) are equipped with XBee modules. Among them, one node functions as the coordinator (C), connected to a PC through a USB-XBee adapter, enabling data exchange, logging, and visualization.

Each sensor is connected to the Arduino via digital or analog input pins and polled at regular intervals. The collected data are transmitted via ZigBee to the coordinator node. Configuring the WSN nodes requires a detailed setup of both hardware and software components. The nodes in this experiment are divided into two types: Arduino Uno microcontroller-based nodes and Raspberry Pi single-board computer (SBC) nodes.

The following configuration elements were defined:

- Key hardware and software setup parameters;
- Wiring diagrams and port assignments;
- Software installation procedures;
- Network configuration parameters.

The Arduino IDE was used to program the microcontrollers. Before uploading the firmware, the XBee module must be temporarily disconnected to prevent UART port conflicts. Once the upload is complete, the module is reconnected.

Packets received from the XBee module over TCP are handled by a secondary Raspberry Pi, where they are unpacked and retransmitted via ZigBee. XBee modules can be configured either on a separate PC using XCTU or on a Raspberry Pi using the Digi-XBee library. After physical connection and configuration of all nodes, the coordinator automatically generates a network topology map, which can be monitored using XCTU. LED indicators were installed on nodes to indicate status (e.g., blinking when data are successfully transmitted). To enhance power stability, voltage regulators and discharge-protected batteries were used. Sensor nodes were tasked with monitoring environmental parameters. Sensors were chosen for their Arduino compatibility, stability, and ease of calibration. Signals from the sensors are polled at regular intervals, processed locally on

the microcontroller, and transmitted via ZigBee to the coordinator, where data are logged and visualized through a dashboard. Sensor reliability was ensured through proper wiring, stable power supply, and robust hardware assembly.

To enable attack modeling and routing behavior analysis, communication limitations were strictly enforced. In WSNs, it is critical to manage node spacing and link stability to avoid the formation of unintended routes. ZigBee modules naturally support distances up to 100 m, which could lead to routing issues.

By combining antenna shielding, reduced transmission power, and strategic physical separation, a controlled WSN environment was created. This setup ensured correct routing behavior, prevented unintended connections, and enabled realistic modeling of network attacks such as the Wormhole attack under near-real-world conditions.

#### 4.5.3. Experimental Results

Modeling the operation of a WSN and implementing a Wormhole attack requires careful planning and controlled placement of network nodes. Proper node deployment is essential for managing network topology, ensuring link quality, and illustrating the overall system behavior—particularly when accounting for the limited communication range of ZigBee and the possibility of artificial route formation.

To mitigate risks associated with linear topology and unintended connections, nodes were strategically deployed across a four-story office building. The experimental setup focused on the third and fourth floors, forming a real-world multi-story testbed that simulated air quality measurements (e.g., CO<sub>2</sub>, CH<sub>4</sub>, and PM levels) within a smart building infrastructure to reflect practical constraints (Figure 17).



**Figure 17.** Smart building infrastructure.

The key objectives of the equipment layout were as follows:

- Ensure that each node communicates only with its immediate neighbors sequentially.
- Prevent direct radio communication between non-active nodes.

- Physically separate the attacker nodes (R4 and R5) to keep them out of each other's signal range.

- Maintain a stable power supply and protect nodes from external physical impacts.

Table 6 provides a detailed overview of the network nodes and their physical placement.

**Table 6.** Network nodes and their placement.

| Nodes                      | Placement                                                                     | Placement Diagram                                   | Power Supply and Stability                                                                    |
|----------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1 Coordinator              | The coordinator (C) is installed in the server room and connected to a laptop | 3-layer:<br>[R5]–[C]–[R0]–[R1]                      | Each node was powered autonomously:<br>- Arduino nodes—via 5 V external batteries or adapters |
| 6 Routers (R0–R5)          | Among which R4 and R5 are Raspberry Pi attacker nodes                         | 4-layer:<br>[R2]–[R3]–[R4]                          | - Raspberry Pi—via 5 V, 2.5 A, stable power adapters                                          |
| The length of the “chain”  | Approximately 100–150 m                                                       | R0 and R1—in administrative offices with open doors | - Coordinator laptop—equipped with a backup battery to ensure uninterrupted operation         |
| The distance between nodes | From 10 to 25 m                                                               |                                                     |                                                                                               |

This placement within the testbed does not guarantee the absence of a direct Zigbee connection between nodes R4 and R5, which is crucial to ensure a realistic simulation of the GSM-based tunnel. To prevent unintended Zigbee communication, XBEE antennas were wrapped in aluminum foil to create directed transmission zones. Zigbee transmission power was minimized using the XCTU tool, and node placement was strategically planned, taking into account physical barriers such as walls, doors, and furniture. Additionally, metal enclosures and partly closed doors, glass partitions, and metallic objects were employed to further influence signal attenuation and propagation.

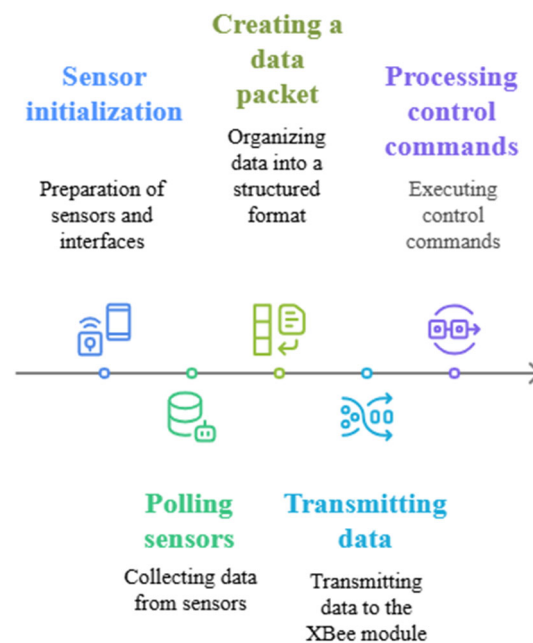
Each node was labeled with its identifier (e.g., R1) and a temporary card containing a brief description. The Zigbee network was formed automatically using a shared PAN ID. The initial network setup was performed via the coordinator node, which managed the registration of nodes and their routing paths. Prior to launching the attack, the network topology was designed to follow a strict linear configuration using XCTU to ensure predictability and control in routing behavior.

Precise node placement was critical for modeling stable network operation, root monitoring, and the simulation of a Wormhole attack. To assess the impact of such an attack within a Zigbee-based WSN, several mitigation and isolation techniques were employed: antenna shielding, reduced transmission power, and physical separation via structural barriers such as walls and floors. These measures successfully enforced a linear network topology, which allowed clear observation of routing disruptions, delays, data loss, and value tampering through the dashboard interface and XCTU following the attack initiation.

Consequently, the hardware–software testbed clearly demonstrated the significant effects of a Wormhole attack on WSN behavior. GSM-based tunneling redirected network traffic from legitimate paths, resulting in data loss, delays, and manipulation. Observable indicators, such as increased latency, data anomalies, and routing changes, were recorded via the dashboard, reinforcing the need for an effective Intrusion Detection and Prevention System (IDS/IPS).

A key element of the system’s functionality is the firmware developed for the Arduino-based WSN nodes. These nodes are responsible for real-time data collection from various sensors and wireless communication via the Zigbee protocol using XBEE modules. The software requirements included reliability, energy efficiency, scalability, and compatibility with selected hardware components. As shown in Figure 18, each node’s firmware is composed of several logical modules that collectively ensure smooth operation within the wireless sensor network.

In a wireless sensor network (WSN) system, the firmware deployed on sensor nodes is fundamental to ensuring the accurate acquisition, processing, and transmission of data. This study employs a hybrid architecture utilizing Arduino and Raspberry Pi platforms, with each platform running a customized software stack optimized for its specific hardware capabilities.



**Figure 18.** Common architecture of firmware.

For Arduino-based sensor nodes, the firmware was developed in C++ using established libraries that support low-level hardware interfacing. Sensor data acquisition is performed via the `analogRead()` function, with collected values transmitted to the XBEE module over the UART interface. Data packets are then encapsulated in Zigbee-compliant frames and forwarded to the network coordinator. Each transmitted message includes a unique node identifier alongside the associated sensor readings, facilitating reliable source identification and data mapping.

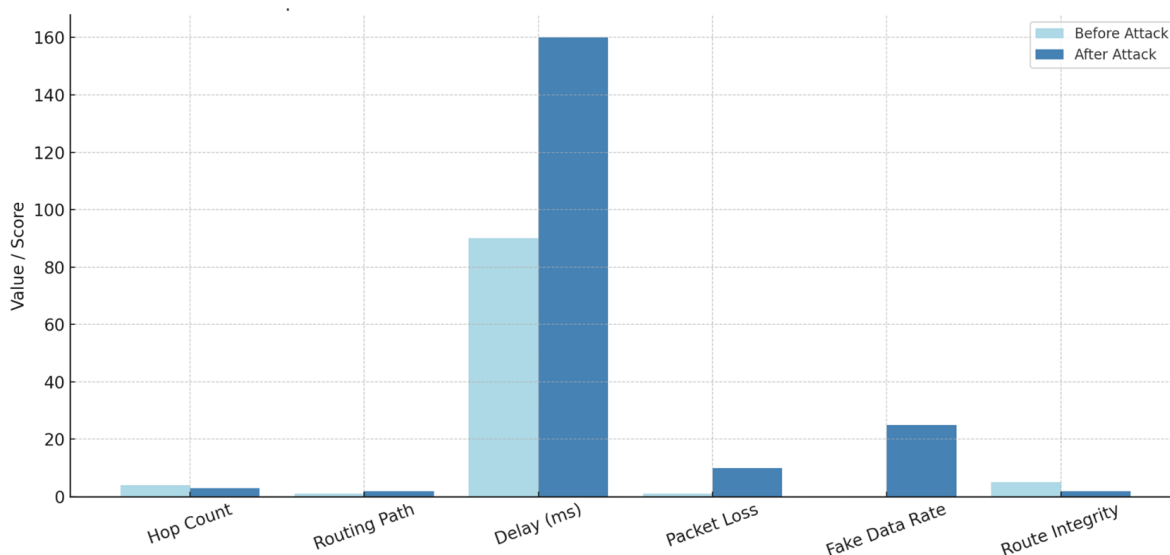
To improve the robustness and resilience of the system, several key mechanisms were integrated into the firmware architecture, including non-blocking execution strategies, timeout management routines, and a watchdog timer to recover from potential system hangs. While acknowledgment (ACK) support was evaluated, it remains only partially implemented due to inherent limitations in the Arduino hardware and library ecosystem.

All Arduino nodes share a unified firmware structure, with variations limited to node-specific parameters such as identification number, sensor type, and pin assignments. Firmware deployment is performed via USB interfaces, during which the connected XBEE modules are temporarily disconnected to prevent transmission conflicts. Debugging and real-time diagnostics are facilitated through the Arduino Serial Monitor and companion software tools, enabling streamlined development and testing workflows. This firmware design ensures modularity, scalability, and consistent operation across diverse WSN deployment scenarios. Raspberry Pi-based nodes provide enhanced functionality, leveraging Python programming and system utilities such as `socket` and `pyserial`. These nodes enable the establishment of tunnel communications, remote data transmission, and the execution of network-oriented attack simulations. The overall software infrastructure is supported by the XCTU tool alongside a custom-developed dashboard. These tools collectively enable efficient monitoring of network routes, real-time visualization of sensor data, and systematic evaluation of attack impacts. The modular architecture of the software allows for flexible system configuration and seamless execution of diverse simulation scenarios, as outlined in Table 7.

**Table 7.** Software components of WSN nodes.

| Component                        | Description                                                                                                                                                                                                                                                                                 |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| XCTU and XBee Configuration      | XBee modules are configured via XCTU by setting PAN IDs, channels, addresses, API modes, and baud rates. The tool allows network connection, topology visualization, and real-time monitoring.                                                                                              |
| Raspberry Pi and Wormhole Tunnel | Raspberry Pi with GSM modules on R4 and R5 simulate attacks. A TCP/IP tunnel using socat or Python scripts forwards ZigBee packets between nodes, appearing as a direct link in the network.                                                                                                |
| Route Changes During Attack      | After the Wormhole tunnel is activated, data are routed in 3 hops instead of 4:<br>Old route: R3 → R2 → R1 → R0 → C;<br>New route: R3 → R4 → R5 → C.<br>This alters routing, bypasses some nodes, and allows the attacker to intercept and manipulate data.                                 |
| Comparative Traffic Analysis     | <ul style="list-style-type: none"> <li>- Before the attack: 4 hops, 80–100 ms delay, accurate data.</li> <li>- After the attack: 3 hops, 120–200 ms delay, 20–30% of data modified by the attacker.</li> <li>- XCTU visualization displays the new R4–R5 link and route changes.</li> </ul> |

Figure 19 shows that after the attacks, the network experiences increased delay, packet loss, and false data transmission, indicating a disruption of route integrity.

**Figure 19.** Comparison of network parameters before and after attacks.

Thus, Figure 19 indicates that a comparative analysis of network parameters reveals a significant difference between pre-elaborate and subsequent attack states. In particular, the hop count increased slightly after the attack, indicating that the data packet had to cross additional intermediate nodes to reach its destination. Although the routing path remained unchanged, adequate changes were observed in the delay time, suggesting that the main communication structure of the system was not directly affected. Increased results of delayed network traffic and resumed starts through the malicious node were observed.

Additionally, the loss of packets increased after the attack, indicating a decline in network reliability. The ratio of fake data increased rapidly due to the attack, increasing the risk of false information infiltrating the system, and data integrity could be compromised. The Route Integrity Metric rejected the post-attack, which reflects breakdown in reliable routing. Overall, the analyzed matrix confirms that Sinkhole and Wormhole attacks have a great negative effect on WSN performance and data trustees. These changes highlight the importance of real-time security monitoring systems and initial detection of malicious activities.

During the decision support phase, appropriate action is taken in the WSN system in response to identified security events. In particular, in a WSN system designed for atmospheric air monitoring, the risks and losses caused by the attack are assessed. This involves evaluating the blockage of monitoring services due to network failure and the financial costs of restoring the infrastructure. Additionally, if the attack continues, the possible future consequences are predicted. System administrators determine potential countermeasures, such as allowing the system to continue operations in autonomous mode or implementing alternative functions. At this stage, all events are logged and documented, which provides a basis for later analysis and investigation after the event. In addition, to increase system security, the security policy is modified and updated. This also includes clearing the roles of each network node and removing excessive privileges; regarding hardware-level security, improving recovery processes after an attack; and applying excesses to important nodes and data storage systems. Overall, this phase enables rapid detection and reaction to attacks in the WSN system and contributes to the prevention of future events. Such an approach is important to ensure the safety of important infrastructure in areas such as manufacturing, healthcare, and environmental monitoring.

Overall, the functioning WSNs developed here greatly improve the accuracy and speed of detection of security events. They enable the rapid identity of both external and internal threats to increase the flexibility of WSN systems against DDOS attacks, data tampering, routing manipulation, and other malicious activities. Additionally, functioning helps prevent interruption and dangers in important applications, such as industrial facilities, medical infrastructure, and urban-scale environmental monitoring systems. Its implementation allows personnel to react rapidly to events, separate their effects, and reduce damage fully, ensuring the reliability and stability of the entire infrastructure. In addition, by depositing data on recorded events, the functioning supports prediction and prevention of future attacks, which contributes to the ongoing improvement in the cyber-physical security level of the WSN system.

## 5. Conclusions

This research introduces a novel cyber-physical approach that integrates hardware-in-the-loop detection with multi-objective optimization for real-time identification of attacks in constrained WSN environments. The paper presents research focused on modeling and detecting Wormhole and Sinkhole attacks in WSNs using real-time capabilities. It combines ZigBee with GSM/GPRS technologies through a modular architecture to deliver strong network attack detection capabilities under normal operational circumstances. The experimental findings established that the new detection approach successfully detected serious network threats, including reduced hop lengths, extended response times, and atypical routing patterns. A multi-stage anomaly detection framework which uses the 3-sigma rule and other techniques allowed the system to immediately detect attacks to protect data integrity and network stability. The developed methodology demonstrated success according to key performance indicators that showed a 40% decrease in hop count and a 27% increase in traffic concentration during Sinkhole attacks. The study delivers a dependable and adaptable and easily scalable security solution which protects WSNs in vital applications

that include smart building and industrial automation along with environmental monitoring. Consequently, the research contributes not only a functional prototype but also lays the groundwork for future studies to develop more energy-efficient and adaptive anomaly detection frameworks suitable for increasingly complex cyber–physical systems. Future work will focus on the incorporation of machine learning techniques to enhance detection accuracy and on extending the framework’s capabilities to encompass multi-vector and coordinated attacks, thereby advancing the resilience of cyber–physical systems.

**Author Contributions:** Conceptualization, T.Z.; Methodology, T.Z.; Software, S.K.; Validation, L.Z.; Formal analysis, L.Z., Y.M. and S.K.; Investigation, T.Z., Y.M. and A.B.; Resources, L.Z.; Data curation, T.Z., A.B., S.K. and N.A.; Writing—original draft, T.Z.; Writing—review and editing, S.K. and L.Z.; Supervision, T.Z. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research has been funded by the Committee of Science of the Ministry of Science and Higher Education of the Republic of Kazakhstan (Grant No. AP19680345).

**Institutional Review Board Statement:** Not applicable.

**Informed Consent Statement:** Not applicable.

**Data Availability Statement:** The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

**Conflicts of Interest:** Authors Tamara Zhukabayeva, Lazzat Zholshiyeva, Yerik Mardenov were employed by the company “International Science Complex Astana”. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

## References

1. Karthikeyan, M.; Revathi, S.T. Approaches to Detecting Threats in Wireless Sensor Networks for Data Transmission Security. In Proceedings of the 2024 International Conference on Advances in Computing, Communication and Applied Informatics (ACCAI), Chennai, India, 9–10 May 2024; pp. 1–7. [\[CrossRef\]](#)
2. Imathiu, G.; Chege, A.; Omamo, A. Security Intrusion Monitoring Model for Internet of Things (IoT) Using Sniffing Tools on Wireless Sensor Networks. *AJSTSS* **2024**, *2*, 51–58. [\[CrossRef\]](#)
3. Zhang, K. A Wormhole Attack Detection Method for Tactical Wireless Sensor Networks. *PeerJ Comput. Sci.* **2023**, *9*, e1449. [\[CrossRef\]](#)
4. Alshomrani, M.; Albeshri, A.; Alturki, B.; Alallah, F.S.; Alsulami, A.A. Survey of Transformer-Based Malicious Software Detection Systems. *Electronics* **2024**, *13*, 4677. [\[CrossRef\]](#)
5. Hanif, M.; Ashraf, H.; Jalil, Z.; Jhanjhi, N.Z.; Humayun, M.; Saeed, S.; Almuhaideb, A.M. AI-Based Wormhole Attack Detection Techniques in Wireless Sensor Networks. *Electronics* **2022**, *11*, 2324. [\[CrossRef\]](#)
6. Ataa, M.S.; Sanad, E.E.; El-Khoribi, R.A. Intrusion Detection in Software Defined Network Using Deep Learning Approaches. *Sci. Rep.* **2024**, *14*, 29159. [\[CrossRef\]](#)
7. Li, H.; Ji, L.; Wang, K.; Liu, S.; Liu, S. Applying the Stackelberg Game to Assess Critical Infrastructure Vulnerability: Based on a General Multi-Layer Network Model. *Chaos* **2024**, *34*, 123127. [\[CrossRef\]](#) [\[PubMed\]](#)
8. Dhama, P.; Prashanth, K. Genetic Algorithm-Based Wormhole Attack Detection in WSN. *Int. J. Sci. Res. Arch.* **2023**, *9*, 795–802. [\[CrossRef\]](#)
9. Al-Ahmadi, S.; Aliady, W.; AlRashedy, A. An Efficient Wormhole Attack Detection Method in Wireless Sensor Networks. In Proceedings of the 2022 26th International Conference on Circuits, Systems, Communications and Computers (CSCC), Crete, Greece, 19–22 July 2022; pp. 240–249. [\[CrossRef\]](#)
10. Naik, R.L.; Jain, S.; Bairam, M. Development of Hybrid Weighted Networks of RNN and DBN for Facilitating the Secure Information System in Cybersecurity Using Meta-Heuristic Improvement. *Wirel. Netw.* **2025**, *31*, 3625–3660. [\[CrossRef\]](#)
11. Zhukabayeva, T.; Pervez, A.; Mardenov, Y.; Othman, M.; Karabayev, N.; Ahmad, Z. A Traffic Analysis and Node Categorization-Aware Machine Learning-Integrated Framework for Cybersecurity Intrusion Detection and Prevention of WSNs in Smart Grids. *IEEE Access* **2024**, *12*, 91715–91733. [\[CrossRef\]](#)
12. Zilberman, A.; Dvir, A.; Stulman, A. IPv6 Routing Protocol for Low-Power and Lossy Networks Security Vulnerabilities and Mitigation Techniques: A Survey. *ACM Comput. Surv.* **2025**, *57*, 1–77. [\[CrossRef\]](#)

13. Sharma, S.; Kumar, P.; Bharti, V. Isolating Sink Hole Attacks in Wireless Sensor Networks Through Trust-Based Mechanism. In Proceedings of the 2024 4th International Conference on Technological Advancements in Computational Sciences (ICTACS), Tashkent, Uzbekistan, 13–15 November 2024; pp. 346–351. [\[CrossRef\]](#)
14. Talukder, M.A.; Sharmin, S.; Uddin, M.A.; Islam, M.M.; Aryal, S. MLSTL-WSN: Machine Learning-Based Intrusion Detection Using SMOTETomek in WSNs. *Int. J. Inf. Secur.* **2024**, *23*, 2139–2158. [\[CrossRef\]](#)
15. Ramkumar, K.; Alzubaidi, L.H.; Malathy, V.; Venkatesh, T.; Kruthika, C.G. Intrusion Detection System in Wireless Sensor Networks Using Modified Recurrent Neural Network with Long Short-Term Memory. In Proceedings of the 2024 International Conference on Integrated Circuits and Communication Systems (ICICACS), Raichur, India, 23–24 February 2024; pp. 1–5. [\[CrossRef\]](#)
16. Kim, T.; Vecchietti, L.F.; Choi, K.; Lee, S.; Har, D. Machine Learning for Advanced Wireless Sensor Networks: A Review. *IEEE Sens. J.* **2021**, *21*, 12379–12397. [\[CrossRef\]](#)
17. Mao, Z.M.; Sekar, V.; Spatscheck, O.; van der Merwe, J.; Vasudevan, R. Analyzing large DDoS attacks using multiple data sources. In Proceedings of the 2006 SIGCOMM Workshop on Large-Scale Attack Defense, Pisa, Italy, 11–15 September 2006; pp. 161–168. [\[CrossRef\]](#)
18. Rehman, A.; Rehman, S.U.; Raheem, H. Sinkhole Attacks in Wireless Sensor Networks: A Survey. *Wirel. Pers. Commun.* **2018**, *106*, 2291–2313. [\[CrossRef\]](#)
19. Birahim, S.A.; Paul, A.; Rahman, F.; Islam, Y.; Roy, T.; Hasan, M.A.; Haque, F.; Chowdhury, M.E.H. Intrusion Detection for Wireless Sensor Network Using Particle Swarm Optimization Based Explainable Ensemble Machine Learning Approach. *IEEE Access* **2025**, *13*, 13711–13730. [\[CrossRef\]](#)
20. Latha, D.J.; Rameswaran, N.; Bharathraj, M.; Vinoth Raj, R. Prevention of Wormhole Attack Using Mobile Secure Neighbour Discovery Protocol in Wireless Sensor Networks. In *IoT Based Control Networks and Intelligent Systems*; Springer: Singapore, 2023; pp. 215–229. [\[CrossRef\]](#)
21. AlShuhail, A.S.; Bhatia, S.; Kumar, A.; Bhushan, B. Zigbee-Based Low Power Consumption Wearables Device for Voice Data Transmission. *Sustainability* **2022**, *14*, 10847. [\[CrossRef\]](#)
22. Das, R.; Bera, J.N. Self-Reconfigurable Partial Mesh Using ZigBee Control Board towards Fail-Proof and Reliable Home Area Networking. *Ad Hoc Netw.* **2024**, *152*, 103327. [\[CrossRef\]](#)
23. Hasan, M.Z.; Rahman, M.A.; Begum, S.; Hossain, M.S.; Ahmed, F.; Gumaei, A. An Efficient Detection of Sinkhole Attacks Using Machine Learning: Impact on Energy and Security. *PLoS ONE* **2025**, *20*, e0309532. [\[CrossRef\]](#)
24. Konatam, S.; Nalluri, S.; Malyala, M.M.; Daiya, H.; Kumar, V.N.; Raju, K.S. A Random Forest-Based Method for Effective and Robust Detection of Wormhole Attacks in Wireless Sensor Networks. In *Intelligent Computing and Communication*; Springer: Singapore, 2025; pp. 461–476. [\[CrossRef\]](#)
25. Teng, Z.; Li, M.; Yu, L.; Gu, J.; Li, M. Sinkhole Attack Defense Strategy Integrating SPA and Jaya Algorithms in Wireless Sensor Networks. *Sensors* **2023**, *23*, 9709. [\[CrossRef\]](#) [\[PubMed\]](#)
26. Al Sukkar, G.; Al-Sharaeh, S. Enhancing Security in Wireless Sensor Networks: A Machine Learning-Based DoS Attack Detection. *Eng. Technol. Appl. Sci. Res.* **2025**, *15*, 19712–19719. [\[CrossRef\]](#)
27. Hassan, S.M.; Mohamad, M.M.; Muchtar, F.B. Advanced Intrusion Detection in MANETs: A Survey of Machine Learning and Optimization Techniques for Mitigating Black/Gray Hole Attacks. *IEEE Access* **2024**, *12*, 150046–150090. [\[CrossRef\]](#)
28. Al-chikh Omar, A.A.R.; Soudan, B.; Altaweel, A. A comprehensive survey on detection of sinkhole attack in routing over low power and Lossy network for internet of things. *Internet Things* **2023**, *22*, 100750. [\[CrossRef\]](#)
29. Radhika, S.; Srikanth, M.; Anand, K.; Saravanan, K.; Southry, S.S. Improving Data Integrity for Gray Hole Attack Detection by Using a Hash Signature Algorithm in WSN. In Proceedings of the 2023 5th International Conference on Smart Systems and Inventive Technology (ICSSIT), Tirunelveli, India, 23–25 January 2023; pp. 619–623. [\[CrossRef\]](#)
30. Bhatti, D.S.; Saleem, S.; Imran, A.; Kim, H.J.; Kim, K.I.; Lee, K.C. Detection and Isolation of Wormhole Nodes in Wireless Ad Hoc Networks Based on Post-Wormhole Actions. *Sci. Rep.* **2024**, *14*, 3428. [\[CrossRef\]](#)
31. Faris, M.; Mahmud, M.N.; Salleh, M.F.M.; Alnoor, A. Wireless Sensor Network Security: A Recent Review Based on State-of-the-Art Works. *Int. J. Eng. Bus. Manag.* **2023**, *15*, 18479790231157220. [\[CrossRef\]](#)
32. Hammi, B.; Idir, Y.M.; Zeadally, S.; Khatoun, R.; Nebhen, J. Is It Really Easy to Detect Sybil Attacks in C-ITS Environments: A Position Paper. *IEEE Trans. Intell. Transp. Syst.* **2022**, *23*, 18273–18287. [\[CrossRef\]](#)
33. De Neira, A.B.; Kantarci, B.; Nogueira, M. Distributed Denial of Service Attack Prediction: Challenges, Open Issues and Opportunities. *Comput. Netw.* **2023**, *222*, 109553. [\[CrossRef\]](#)
34. Khalaf, B.A.; Mostafa, S.A.; Mustapha, A.; Mohammed, M.A.; Mahmoud, M.A.; Al-Rimy, B.A.S.; Marks, A. An Adaptive Protection of Flooding Attacks Model for Complex Network Environments. *Secur. Commun. Netw.* **2021**, *2021*, 5542919. [\[CrossRef\]](#)
35. Wang, H.; Huang, X.; Wu, Y. GD3N: Adaptive Clustering-Based Detection of Selective Forwarding Attacks in WSNs under Variable Harsh Environments. *Inf. Sci.* **2024**, *665*, 120375. [\[CrossRef\]](#)

36. Pirayesh, H.; Zeng, H. Jamming Attacks and Anti-Jamming Strategies in Wireless Networks: A Comprehensive Survey. *IEEE Commun. Surv. Tutor.* **2022**, *24*, 767–809. [\[CrossRef\]](#)
37. Elgindy, M.M.; Elghamrawy, S.M.; El-Desouky, A.I. Proposed Mitigation Framework for the Internet of Insecure Things. *Mansoura Eng. J.* **2023**, *48*, 3–10. [\[CrossRef\]](#)
38. Zahra, F.; Jhanjhi, N.Z.; Khan, N.A.; Brohi, S.N.; Masud, M.; Aljahdali, S. Protocol-Specific and Sensor Network-Inherited Attack Detection in IoT Using Machine Learning. *Appl. Sci.* **2022**, *12*, 11598. [\[CrossRef\]](#)
39. Swami, S.; Singh, P.; Chauhan, S.S. An Integrated Rule-Based and Machine Learning Technique for Efficient DoS Attack Detection in WSN. In Proceedings of the 2024 2nd International Conference on Disruptive Technologies (ICDT), Greater Noida, India, 15–16 March 2024; pp. 847–851. [\[CrossRef\]](#)
40. Khan, S.; Khan, M.A.; Alnazzawi, N. Artificial Neural Network-Based Mechanism to Detect Security Threats in Wireless Sensor Networks. *Sensors* **2024**, *24*, 1641. [\[CrossRef\]](#) [\[PubMed\]](#)
41. Alshehri, A.H. Wormhole Attack Detection and Mitigation Model for Internet of Things and WSN Using Machine Learning. *PeerJ Comput. Sci.* **2024**, *10*, e2257. [\[CrossRef\]](#) [\[PubMed\]](#)
42. Shahid, H.; Ashraf, H.; Jhanjhi, N.Z.; Zam Zam, Q.A. Reshaping Cybersecurity of Wireless Sensor Networks Using Energy-Optimized Approach Against Wormhole Attack. In *Advances in Information Security, Privacy, and Ethics Book Series*; IGI Global: Hershey, PA, USA, 2024; pp. 125–172. [\[CrossRef\]](#)
43. Seidu, R.; Salifu, A.M.S.; Ansuura, J.A. A Genetic Algorithm with a Trust Model Function for Detecting Sinkhole and Wormhole Nodes in Wireless Sensor Networks. *Trans. Eng. Comput. Sci.* **2024**, *12*, 30–44. [\[CrossRef\]](#)
44. Sun, Y.; Chen, Y. Detection of Wormhole Attacks in Wireless Sensor Networks Based on Anomaly Detection Algorithms. In Proceedings of the 2022 2nd International Conference on Consumer Electronics and Computer Engineering (ICCECE), Guangzhou, China, 14–16 January 2022; pp. 777–782. [\[CrossRef\]](#)
45. Yuan, J.; Yan, B. A Detection Method of Wormhole Attack in Power Communication Sensor Networks Based on Hops. *Web Intell.* **2022**, *21*, 115–125. [\[CrossRef\]](#)
46. Al-Ahmadi, S. A Novel Energy-Efficient Wormhole Attack Prevention Protocol for WSN Based on Trust and Reputation Factors. In Proceedings of the 11th International Conference on Sensor Networks (SENSORNETS), Online, 7–8 February 2022; pp. 191–201. [\[CrossRef\]](#)
47. Sridevi, S.; Anandan, R. RUDRA—A Novel Re-Concurrent Unified Classifier for the Detection of Different Attacks in Wireless Sensor Networks. In *Intelligent Computing in Engineering*; Springer: Singapore, 2020; pp. 251–259. [\[CrossRef\]](#)
48. Lakshmi Narayanan, K.; Santhana Krishnan, R.; Golden Julie, E.; Harold Robinson, Y.; Shanmuganathan, V. Machine Learning Based Detection and a Novel EC-BRTT Algorithm Based Prevention of DoS Attacks in Wireless Sensor Networks. *Wirel. Pers. Commun.* **2021**, *127*, 479–503. [\[CrossRef\]](#)
49. Mondal, K.; Yadav, S.; Pal, V.K.; Singh, A.P.; Singh, M. Detecting Sinkhole Attacks in IoT-Based Wireless Sensor Networks Using Distance from Base Station. *Int. J. Inf. Syst. Model. Des.* **2022**, *13*, 1–18. [\[CrossRef\]](#)
50. Ali, T.S. MANDS: Malicious Node Detection System for Sinkhole Attack in WSN Using DRI and Cross Check Method. In Proceedings of the International Conference on Computational Intelligence in Pattern Recognition, Howrah, India, 23–24 April 2022; Springer: Singapore, 2022; pp. 511–519. [\[CrossRef\]](#)
51. Ambika, N. Improved Cross-Layer Detection and Prevention of Sinkhole Attack in WSN. In *Handbook of Research on Innovations and Applications of AI, IoT, and Cognitive Technologies*; IGI Global: Hershey, PA, USA, 2021; pp. 514–527. [\[CrossRef\]](#)
52. Ahmad, R.; Wazirali, R.; Abu-Ain, T. Machine Learning for Wireless Sensor Networks Security: An Overview of Challenges and Issues. *Sensors* **2022**, *22*, 4730. [\[CrossRef\]](#)
53. Zhukabayeva, T.K.; Desnitsky, V.A.; Mardenov, E.M. A Technique for Collection, Preprocessing and Analysis of Data in Wireless Sensor Networks. *News Natl. Acad. Sci. Repub. Kazakhstan Phys.-Math. Ser.* **2024**, *2*, 163–176. [\[CrossRef\]](#)
54. Shehzadi, K.; Abbas, T.; Zainab, A.; Li, H. A survey of intruder detection in smart grid systems: Comparative analysis of rule-based, machine learning and deep learning. *Int. J. Secur. Netw.* **2025**, *20*, 67–92. [\[CrossRef\]](#)
55. Umar, M.M.; Khan, S.; Ahmad, R.; Singh, D. Game Theoretic Reward Based Adaptive Data Communication in Wireless Sensor Networks. *IEEE Access* **2018**, *6*, 28073–28084. [\[CrossRef\]](#)
56. Alrajeh, N.A.; Khan, S.; Lloret, J.; Loo, J. Artificial neural network based detection of energy exhaustion attacks in wireless sensor networks capable of energy harvesting. *Ad-Hoc Sens. Wirel. Netw.* **2014**, *22*, 109–133.
57. Gandhi, K.I.; Kannan, G. Development of an Energy-Efficient Patient Monitoring System Using RSSI-Based Wireless Sensor Network with AODV and ZigBee Technology. *J. Nano-Electron. Phys.* **2024**, *16*, 06005-1–06005-5. [\[CrossRef\]](#) [\[PubMed\]](#)
58. ASoussi, A.; Zero, E.; Sacile, R.; Trincherro, D.; Fossa, M. Smart Sensors and Smart Data for Precision Agriculture: A Review. *Sensors* **2024**, *24*, 2647. [\[CrossRef\]](#)
59. Majhi, A.A.K.; Mohanty, S. A Comprehensive Review on Internet of Things Applications in Power Systems. *IEEE Internet Things J.* **2024**, *11*, 34896–34923. [\[CrossRef\]](#)

60. Stanco, G.; Navarro, A.; Frattini, F.; Ventre, G.; Botta, A. A comprehensive survey on the security of low power wide area networks for the Internet of Things. *ICT Express* **2024**, *10*, 519–552. [[CrossRef](#)]
61. Howitt, I.; Gutierrez, J.A. IEEE 802.15.4 low rate—Wireless personal area network coexistence issues. In Proceedings of the 2003 IEEE Wireless Communications and Networking, WCNC 2003, New Orleans, LA, USA, 16–20 March 2003; Volume 3, pp. 1481–1486. [[CrossRef](#)]

**Disclaimer/Publisher’s Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.